

Guided Editing - Product Spec

Design & UX Principles

- Gamified progress (via CV Score).
- Simplicity triumphs all. Maximum simplicity and friction reduction. Simplify, simplify, simplify. Every detail, button, text, interaction ... must be dead simple, intuitive, frictionless
- Design with monetization in mind
 - We try to design & offer the best user experience, then find smart ways to monetize our features
- Visual & Interaction Guidelines:
 - Clean and professional: modern, minimalist interface aligning with industry standards.
 - Consistent colors and fonts: primary colors soft cyan (#E0F7FA), gentle mint green (#E6F4EA), light sky blue (#E3F2FD); typography professional yet friendly.
 - Consistent button styles: rounded corners, clear actionable text
 - Consistent typography and visual hierarchy established in previous steps
 - Intuitive interactions: clear affordances, intuitive buttons, and CTAs; maintain consistency across interactions.

UI/UX Design Patterns

```
### Color Scheme
- **Primary**: Blue (#0288D1, #3B82F6)
- **Success**: Green (#4CAF50, #22C55E)
- **Warning**: Orange (#F59E0B)
- **Error**: Red (#EF4444)
- **Gray Scale**: #111827, #6B7280, #E5E7EB, #F9FAFB
```

Overview

The **Guided Editing** step of the OkBuddy CV editor is a dynamic, AI-powered resume builder interface modeled after Resume.io's section-based UX/UI, but enhanced to deliver a 10× better experience. It allows users to edit their resume section-by-section in any order (no rigid step-by-step wizard), with intelligent AI assistance at every turn. Users can freely rearrange sections via drag-and-drop and see a real-time PDF preview update instantly on the right side.

The design follows the *industry-standard Dennis Schröder CV template*: a clean layout with Contact Info centered at the top, followed by Summary, Work Experience, Skills, and Education sections.

Each section is individually editable, renameable, and removable, with AI features like targeted keyword suggestions, auto-completion, and one-click content improvements integrated directly into the workflow.

Key Principles and Improvements

- **Flexible Section Editing:** All resume sections are accessible and editable at any time – no enforced linear sequence. This non-linear flow empowers users to focus on sections in any order without a forced “wizard” progression. Sections can be easily added, removed, renamed, collapsed, or reordered via intuitive 6-dot drag handles (mirroring Resume.io’s approach).
 - Unlike linear wizards, OkBuddy’s interface enables on-the-fly reordering and editing of sections with no friction, plus the option to collapse sections to reduce clutter, giving users unprecedented control over their workflow.
- **Persistent Live Preview:** A WYSIWYG PDF preview of the resume is always visible in the right panel, updating in real time with every change. The preview uses the exact Dennis Schröder template styling (centered name and contacts, section headings, fonts, spacing) to show *exactly* how the final resume will look. As users type or re-order content, the preview reflects those updates within milliseconds, ensuring users don’t have to guess the outcome.
 - The preview is interactive – users can click on any section in the preview to jump to editing that section on the left, bridging the gap between form and output (Resume.io’s preview was passive; OkBuddy’s is actionable).
 - Changes are applied instantly without needing a manual refresh , making the editing experience truly WYSIWYG and eliminating surprises at download time.
- **AI-First Writing Assistance:** Every section includes AI-driven features to minimize manual typing and help users create high-quality content effortlessly. From GPT-powered autocompletion (suggesting text after a few words) to one-click “Generate” and “Improve” buttons, the editor proactively helps craft each section. For example, typing just a few words can trigger Smart Compose-style suggestions to finish the sentence. Users can accept these suggestions with a keystroke, dramatically reducing typing effort.
 - OkBuddy transforms resume writing with multiple AI solutions: it can auto-generate entire section drafts, provide example-based sentence starters, offer drag-and-drop bullet point libraries, and even rewrite or polish content on demand. This far exceeds Resume.io’s limited AI toolkit (which was mostly static examples or a basic summary generator) . Pain points like writing long job descriptions are addressed by AI pre-filling content or offering curated examples, so the user never faces a blank box.
- **Job Description Targeting:** The editor fully implements Resume.io’s “arrow hitting target” concept of tailoring the resume to a specific job. Users can input a target job

description (via a prompt or “Target Job” button), and the AI will analyze it for important keywords and skills. Each section then displays suggestions (often via a target icon  on the section header) for keywords or phrases to incorporate, drawn directly from the job posting . For instance, if a job asks for “project management” or “SQL,” the Work Experience and Skills sections will prompt the user to include those terms if missing.

- OkBuddy’s implementation goes beyond showing keywords in a pop-up – it can automatically integrate them. Users can click “Apply All” to have the AI seamlessly weave all suggested keywords into the relevant sections in one go, or accept suggestions individually. This ensures the resume is highly targeted to the job **without** manual copy-pasting (Resume.io requires manually copying suggested keywords into your text) . The result is a much higher ATS-compatibility and relevance with minimal user effort.
- **Continuous Autosave and Error Prevention:** All changes are persistently saved in real time, so the user never has to worry about losing progress. Every keystroke or action triggers an autosave (debounced to every few seconds or on field blur) to the user’s profile. A “Saving...” indicator can briefly display and then confirm “All changes saved.” Input validation catches errors immediately – e.g. invalid email formats, date inconsistencies, or overly long sections – and provides inline feedback. The system also suppresses empty sections from the preview (sections with no content won’t appear in the PDF) to avoid blank headings.
 - Robust autosave and smart validation mean users can focus on content while the tool guards against mistakes. For example, if an email is malformed or a date range is illogical, the field is highlighted with an error message and suggestions to fix it (instead of silently allowing a bad entry). Compared to typical builders, which might let errors slip through or only validate at the end, OkBuddy ensures quality and consistency continuously.

Below, we detail the UX flow, visual hierarchy, component behaviors, and AI logic for each part of the Guided Editing interface.

UX Flow and Layout

Two-Column Editor Layout

The Guided Editing interface is split into two primary panels:

- **Left Panel – Section Editor:** This is a scrollable panel containing all the resume sections (Contact Info, Summary, Work Experience, Skills, Education, etc.) one after another. Each section is presented as a collapsible card with its own header and content fields. Users can click into any field to edit text, with changes appearing immediately in the preview. The left panel’s design emphasizes clarity and guidance – section headings are prominent, and form fields are neatly organized with labels or placeholder text. The

user can navigate freely by scrolling or collapsing sections to jump around. There is no prescribed order of completion, aligning with the non-linear editing principle.

- **Right Panel – Live PDF Preview:** The right panel displays a live-rendered PDF preview of the resume, using the chosen template (Dennis Schröder design). It mirrors the left panel's content exactly, updating in real time as the user makes changes. The preview panel has a fixed width (matching standard A4/Letter size at a readable zoom level) and occupies roughly 40% of the screen width, while the left panel takes ~60% for comfortable editing. A vertical divider separates the two panels. The preview uses actual PDF generation or high-fidelity HTML/CSS that matches the PDF output, ensuring what-you-see-is-what-you-get. It is **persistently visible** on desktop; on smaller screens (tablet/mobile), the UI may allow toggling between edit view and preview or stacking them vertically for space.

Visual Hierarchy: The left panel has a light background with distinct section containers, so users can easily distinguish one section from the next. Section headers might have a slightly shaded background or bold text to set them apart from the fields. Within each section, form labels and inputs are cleanly laid out (e.g., label above input or placeholder within input for concise design). Important actions (like “Add Work Experience” or AI suggestion buttons) are emphasized with icons or subtle color highlights. Meanwhile, the right preview panel often draws the eye with the finished look of the resume – key text like section titles and the user's name are in larger fonts, mirroring how the final reader will see it. The user's name at the top center is likely the first thing one notices on the preview, reinforcing that focus. By constantly reflecting changes, the preview reduces the cognitive load – the user can visually verify hierarchy (e.g., that Work Experience is more prominent than Skills) without needing to imagine it.

****Overall Layout Structure:****

- ****Left Panel (CV Editor):**** 60% of screen width, scrollable
- ****Right Panel (CV Preview):**** 40% of screen width, fixed position
- ****Vertical Divider:**** Separates panels with resize capability
- ****Responsive Behavior:**** Stack vertically on mobile/tablet

****Critical Layout Principle:****

- ****One Scroll Rule:**** Only the CV Editor panel (left side) scrolls
- ****Fixed Preview:**** CV Preview is hard-fixed, zoomed out to ensure constant overview of 1 page CV at all times

```
- **Preview Isolation:** When users scroll, only the CV Editor panel content moves; CV Preview remains stationary for constant visibility
```

Guided Editing User Journey

```
#### Progressive User Journey
```

```
**Designed User Flow:**
```

1. ****Summary (Empty)**** → Guidance to Work Experience
2. ****Work Experience**** → Fill in position and company
3. ****AI Assistance**** → Generate bullets and descriptions
4. ****Return to Summary**** → AI can now generate quality summary based on experience data

1. **Initial State:** When the user enters the Guided Editing step, all default resume sections (Contact Info, Summary, Work Experience, Skills, Education) are already present in the left panel, in the template's recommended order. If this is a new resume, these sections are initially empty (with helpful placeholders and hints). For example, the Summary text area might display a placeholder like *"e.g. Passionate [Your Profession] with X years of experience..."*. The Work Experience section might show a prompt or button to add the first job. The right panel initially shows the template with section headings but no content, or if empty sections are hidden by default, it might just show the header (name/contact) and a placeholder message for other sections. The interface may highlight or briefly animate the first suggested section to start with (e.g., a subtle glow around Contact Info or Summary) but this is just a suggestion – the user is free to click any section.
2. **Free Navigation & Editing:** The user begins editing by selecting any field in any section. For instance, they might start with *Contact Info* by typing their name, or jump directly to *Work Experience* to add a job. There are no "Next" or "Previous" buttons forcing a sequence; instead, each section stands on its own. The user can scroll or collapse others to focus on one. As soon as they start typing in a field, that field's container highlights (to show focus) and the corresponding content in the preview might highlight as well (e.g., editing the Summary could highlight the Summary paragraph on the preview for easy reference). The user sees their input reflected in the preview almost immediately. For example, typing their name in the Name field will update the large centered name on the preview in real time. This continuous feedback assures the user of what the final output looks like and encourages iterative refinement.
3. **AI Assistance & Suggestions:** While editing, the user encounters various AI-driven helpers:

- If the user pauses after typing a few words (say, they type “Led a team of” in a bullet point and pause), an **inline suggestion** in faint text might appear (e.g., “ 5 software engineers to develop a new feature”), which the user can accept with a keystroke (Tab or right-arrow) or continue typing to ignore. This is the **Smart Compose** feature offering to complete the sentence.
 - In empty fields, **AI suggestion buttons or placeholders** are visible. For example, an empty Summary section might show a “**Generate Summary with AI**” button. Clicking it triggers the AI to draft a professional summary based on the user’s existing data (or a general template if no data yet). The generated text is then inserted into the Summary field and displayed on the preview. The user can accept it as-is or edit it. Similarly, in Work Experience, when a new job entry is created and the user fills in “Job Title” and “Company”, a “**Suggest Descriptions**” prompt may appear, offering AI-generated example bullet points for that role.
 - Each section header has an **AI toolkit icon** (for example, a magic wand ✨ or a target 🎯 icon depending on context) that when clicked, opens a dropdown of AI options for that section (if the section has multiple AI functions). For instance, Work Experience’s AI menu might include: “*Generate bullets for this role,*” “*Improve all bullet phrasing,*” “*Add missing keywords,*” etc. The user can thus manually invoke AI actions at the section level.
 - If a target job description has been provided (see next step), the sections might also display **targeted suggestions** automatically. For example, a small notification under Skills might read “Suggested skill: SQL (from job posting)” with a one-click “Add” button.
4. **Reordering & Organization:** At any point, the user can reorganize the resume sections to their liking. For example, a student user might drag the *Education* section above *Work Experience*. To do this, the user hovers over the section header; a 6-dot handle appears to the left of the title (the cursor changes to a move icon). The user drags the section up or down – as they drag, a placeholder line indicates the new position. Dropping it will instantly reflow the sections in the left panel *and* update the right-panel preview to the new order (the Education heading and content will now appear above Work Experience on the preview). This reordering can be done at any time and as often as needed . There’s no need to start over or follow a rigid template – all changes are fluid. Similarly, within multi-entry sections like Work Experience, the user can reorder the individual entries (jobs) by dragging them. Each job entry card has its own small drag handle, enabling, for instance, the job at *Company B* to be dragged above the job at *Company A*. The preview will update so that Company B’s job appears first in that section. This fine-grained drag-drop behavior applies to Education entries as well. **(See “Drag & Drop behaviors” below for more details.)**

5. **Job Description Tailoring (Optional flow):** If the user wants to tailor their resume to a specific job, they can invoke the **Job Targeting** feature. Typically, a button labeled “Target My Resume to a Job” or an icon of an arrow hitting a target is available at the top of the editor or in each section’s header. Clicking it brings up a dialog where the user can paste a job description text or upload a job posting. Once submitted, the AI analyzes this input. The UI then highlights each section with any suggested changes: e.g., the Summary section header might display a small badge “🎯+2” indicating two keyword suggestions, Skills might show “🎯+3”, etc. The user can click these indicators to see details. For example, clicking the Summary’s suggestion badge could show: *“The job description mentions ‘project management’ and ‘data analysis’ which are not in your summary. Consider adding them.”* The user can press **“Incorporate All”** – the AI will then seamlessly edit the Summary text (either by appending a relevant phrase or rewriting a sentence) to include those terms. The modifications are highlighted for a moment (to draw attention), then become normal text. The user retains full control and can undo or tweak as needed. This process is repeated per section: in Work Experience, perhaps the job posting mentions specific tools, so the AI suggests adding “Agile methodology” to one of the bullets – the user can accept individually or apply all suggestions in one click. The result is a highly customized resume aligned to the job criteria. Throughout, the right-hand preview updates so the user sees their resume’s match improve. (Optionally, a “match score” or progress bar could show percentage of keywords matched – e.g., *Resume now matches 90% of target job keywords* – giving positive feedback to the user).
6. **Continuous Save and Feedback:** The user’s input is continuously saved. As they proceed through the above steps, a small status text (e.g., in the page footer or header) might briefly show “Saving...” and then “All changes saved”. If the internet connection drops or the save fails, a warning icon with tooltip might appear (but the app will keep the data locally and retry). Validation checks run in the background. For example, if the user leaves the phone number field empty or in a wrong format, the Contact Info section header might show a tiny alert icon. Hovering it informs “Phone number format invalid” or “Missing phone number” depending on rules (the user can choose to ignore if phone is optional, but the system will gently remind). Similarly, if a date field in Work Experience is left blank, an inline prompt might say “End date not specified – use ‘Present’ if currently employed.” None of these validations block the user from continuing; they simply surface potential issues in real-time so the user can correct them at their convenience. This ensures that by the time the user finishes editing, the resume is consistent and error-free, without a last-minute scramble to fix errors.
7. **Finalizing:** Once satisfied, the user can proceed to download or share the resume (outside the scope of this specific editing step). The Guided Editing UI will remain accessible for further tweaks at any time; every change has been saved, and the user can return later to adjust content or tailor to a new job by pasting a different description. The combination of intuitive editing, visual preview, and AI enhancements significantly

accelerates the resume creation process while improving quality – delivering an experience that is leaps ahead (“10× better”) of traditional resume builders.

Left Panel – Section Editor Details

The left panel organizes resume content into **sections**, each corresponding to a logical grouping on the resume. By default, the Dennis Schröder template includes the following required sections (default in order): **Contact Information**, **Summary**, **Work Experience**, **Skills**, and **Education**. Users can also add **custom sections** as needed (e.g., Projects, Certifications), which behave similarly. Each section in the editor is represented as a card with a header row and a content area. Below is a detailed breakdown of section components and behaviors:

Section Header Controls

Every section header contains:

- **Section Title Label:** The name of the section (e.g., “Work Experience”). This label is editable – clicking or tapping on it transforms it into an inline text field (or opens a small input) allowing the user to rename the section. For instance, “Work Experience” could be renamed to “Professional Experience” if the user prefers. Pressing Enter or blurring the field saves the new name. The updated name instantly appears on the preview as the section heading (except for Contact Info, which doesn’t display a heading in the template). Default section titles are provided (and localized, if needed) but the user has full control to customize them.
 - **Validation:** Section titles cannot be left empty – if a user clears it entirely, it will revert to the default name or prompt “Section title required.” They are also constrained to a reasonable length (if a title is over 30 characters, the UI may truncate in the preview or shrink the font).
- **Drag Handle (Reordering):** On the left side of the header, a grip icon with six dots (••••••) is shown (visible on hover or focus to keep the UI clean). Users can grab this handle to drag and drop the entire section to a new position in the left panel. As described in the UX flow, dragging a section moves it along the list; other sections shift to indicate where it will drop. Releasing the mouse (drop) will reorder the section. The right panel’s preview reorders accordingly. The drag handle is disabled (grayed out or not shown) for any sections that cannot be moved.
 - In this template, **Contact Information** is fixed at the top (since the resume design expects it at top-center), so the contact section’s drag handle is absent.
 - All other sections can be rearranged freely. The reordering operation does not require confirmation – it’s immediate and undoable by simply dragging again. (**Internal behavior:** the underlying data structure updates the section order and triggers a re-render of the preview).

- **Collapse/Expand Toggle:** Each section can be collapsed to hide its content fields. On the far right of the header, a chevron icon (▼ or ►) indicates the current state: ▼ if expanded (click to collapse), ► if collapsed (click to expand). Collapsing a section instantly hides all its input fields and sub-content, leaving only the header row visible. In collapsed state, the section's header may also show a brief summary of content for quick reference.
 - For example, the Skills section header when collapsed could display something like "Skills: Java, Python, Team Leadership..." truncated, or the Summary section might show the first sentence of the summary. This way, even when collapsed, the user has context about what's inside.
 - Collapsing is purely for the editor convenience – it does not affect the preview (the preview always shows all content of non-empty sections). Users might collapse sections they consider done or not immediately working on, to reduce scrolling. Expanding brings the fields back. The collapse state is remembered during the session but not necessarily persisted once the user leaves (optional: we could persist it in local settings).
- **Section Menu (Delete/Add):** A context menu or inline icons allow deletion or addition of sections:
 - A **Delete (Trash) Icon** is available on sections that are removable. Clicking delete will prompt a confirmation like "Are you sure you want to delete the *Education* section? This will remove all entries under Education." If confirmed, the section is removed from the left panel, and the preview also removes that section entirely. (Deletion is irreversible via UI, but the user could re-add the section manually; we may also implement an undo for a few seconds after deletion as a safety net). By default, core sections like Work Experience, Skills, etc., are removable in case the user doesn't need them (for instance, a user with no work experience might delete that section for now). The **Contact Info & Work Experience** section are not deletable – that's mandatory for any resume. If the user deletes a section and later wants it back, they can use the "Add Section" function (described below).
 - An **Add Section** button or menu is typically located at the bottom of the sections list (not in each header, since adding is a global action). Clicking "Add Section" opens a small panel of section types to choose from: e.g., "Projects, Certifications, Languages, Awards, [Custom Section]." The user selects one, which then appears as a new section card at the bottom of the left panel (or directly below a selected section if we allow inserting in between). The new section has a default title (which can be edited) and appropriate default fields (e.g., a Projects section might behave like Work Experience with Title/Description fields for each project, while a Languages section might behave like Skills with a list of languages and proficiency). The user can then fill it in, reorder it, etc. These

custom sections also have the same header controls (rename, drag, collapse, delete) as standard sections.

- OkBuddy supports full customization of sections on the fly, whereas Resume.io offered a set list at the bottom that could be added. Here it's seamless and integrated with AI as well – if a custom section is added, the AI engine will still provide writing help relevant to that section's content (for instance, if "Certifications" is added, AI might suggest common certifications in the user's field once it knows the field).
- **AI Suggestions Indicator:** (Optional in header) If the user has provided a job description to or there are AI recommendations for that section, the section header might display an icon or badge. For example, the 🎯 icon might appear if job-tailoring suggestions are available, or a small "✨" if AI has an improvement tip (like "simplify language in this section" or "3 suggestions available"). This indicator invites the user to click for more info (as described in the targeting flow). It is subtly colored (e.g., blue or green) to catch attention but not overwhelm the header. If multiple suggestion types exist, the icon could have a count or different icons for keywords vs. writing style suggestions. For simplicity, one unified icon with a count can open a modal listing all suggestions for that section.

Section Controls & Navigation – Compared to Resume.io, which allowed renaming and reordering sections, OkBuddy adds more robust control: collapsible sections to declutter the view, real-time suggestion indicators, and an interactive preview link. These improvements make navigation and organization much more user-friendly. Users can tailor the editing environment to their needs (focus on one section at a time, reorder instantly, etc.) without the rigid structure some builders impose. This free-form yet guided approach is especially helpful for creative control and reduces the cognitive load of editing a large document.

Section Content and Fields

Each section's content area contains input fields appropriate to that section's information, along with any relevant UI elements like add-buttons for entries or AI action buttons. Below, we break down each default section's structure, behaviors, and AI enhancements in detail, including their empty vs filled states, validation rules, and how they reflect on the preview.

Contact Information Section

Purpose: Collects the user's personal details displayed at the top of the resume. In the Dennis Schröder template, contact info is center-aligned at the top, with the name in a large font and contact details (email, phone, location) typically in a smaller font below the name.

Fields:

- *Name* (Full Name) – Text input. This field is for the user's name, which will be displayed prominently centered at the very top of the resume (usually in a larger font size). If

possible, we'll treat this as a single line input for simplicity (assuming users will enter their full name as they want it shown).

- *Email* – Text input. Used for the email address, shown in the contact info line.
- *Phone* – Text input. For phone number.
- *Location* – Text input. For the user's city/state/country or other location info (depending on user preference, e.g., "Ho Chi Minh City, Vietnam").
- (Optional) *LinkedIn/Website* – We did not list this in the template explicitly, but some users may want to add a LinkedIn URL or personal website. If the Dennis template supports it, we can include an optional field for a web link. For now, we'll focus on the four main items above, as asked.

These fields are typically arranged in a simple form layout (perhaps two columns in the editor if space allows: e.g., Name on one line by itself, then Email and Phone side by side, then Location, or each on its own line). On the preview, they will be formatted all on one centered block: the Name appears on one line, and below it the other details might appear separated by bullet dots or vertical bars (e.g., "email@example.com • +1 234 567890 • Ho Chi Minh City, Vietnam") – exact formatting as per template design.

Default/Empty State: When starting, if the user profile has no preloaded data, these fields are empty. We use placeholders to guide the user, for example:

- Name: "Full Name" (maybe greyed example "John Doe").
- Email: "email@example.com".
- Phone: "(123) 456-7890" or "+84 90 1234 567".
- Location: "City, Country".

If a field is empty, it simply won't appear on the preview (e.g., if phone is empty, it's omitted from the contact line entirely). The section header (Contact Info) doesn't show on preview at all (no title "Contact Info" is printed), it's just a conceptual grouping in the editor. Thus, if all contact fields were empty (which we discourage), the preview top would essentially be blank (aside from template graphics if any).

Validation & Error Handling

- The Name field is required for a valid resume (one should not produce a resume with no name). If the user leaves it blank or erases it, we'll show an inline error like "Name is required." We might also lightly highlight the section header with an alert icon until it's filled. We ensure no invalid characters (it should be text only; numbers or symbols likely a mistake – we can warn if those are present, though we won't forbid since some names have hyphens, etc., but if a user types "John Doe 123", we can gently ask if that's

intended).

- Email: Must be a valid email format (we check for “@” and a plausible domain). On blur, if the format looks wrong, a red error message “Please enter a valid email address” appears under the field. If the user tries to leave it blank, that might be okay (email is often expected on a resume but not absolutely mandatory; we can allow blank but perhaps mark it as “recommended”). If blank, no error, just no email on resume. If invalid and user ignores the error, we might show a warning icon in section header as well.
- Phone: Similar approach – not strictly required, but recommended. We can validate allowed characters (digits, plus, parentheses, dashes, spaces). If something clearly off is entered (e.g., “ABC123”), show “Invalid phone number format.” We can auto-format common patterns (for example, as the user types a US number, auto-add parentheses/dashes) to assist.
- Location: Free text, no strict validation (except maybe length). It’s optional; no error if empty. If filled, ensure it’s not insanely long (e.g., user pasting an entire address – we might truncate or advise to keep it short). Possibly, we can integrate a location autocomplete (Google Maps API) to help standardize city names, but that’s a stretch goal. At minimum, no spelling or grammatical check here, since it’s often just proper nouns.
- LinkedIn/Website (if present): Validate URL format (must contain http:// or domain etc), or simplified to just “username” if we append the site domain automatically.

Once filled, the contact info immediately shows on the preview centered and well-formatted. The name likely appears in larger bold font, and the contact details line appears below. If the user updates any field (say edits their phone number), the preview text updates immediately.

Summary Section

Purpose: The Summary (also known as Professional Summary or Profile) is a brief opening statement (typically 2-4 sentences) summarizing the candidate’s experience, skills, and goals. It appears below the contact info (with a heading like “Summary” or a stylized separator, depending on the template). It is also not drag-able (there is no drag-and-drop button)

Field Type: A multi-line text area, allowing rich text or at least multiple sentences. By default, let’s assume plain text input (no formatting except line breaks if any, but usually it’s one continuous paragraph). It can expand vertically as the user types more lines (up to a limit). The editor shows a character or word count indicator and possibly guidance text.

Default/Empty State: If the user hasn’t entered anything, the Summary section shows a placeholder or helper text inside the text area: *“Write a brief summary of your professional background... (e.g. “Results-driven marketing specialist with 5+ years of experience...”)”*.

- If completely empty, the right-side preview will not display a “Summary” heading or any blank lines (the section is effectively removed from the PDF until content is added).
- The editor also provides an obvious **AI helper** in the empty state: a button labeled “Generate Summary” or a suggestion like “*Let AI write a draft for you*” right within the section UI. This could appear as a ghost button overlay in the empty text box or a separate small button next to it.

Input Guidelines: We recommend keeping the summary concise (for Dennis template likely a short paragraph). Beneath the text box, we display a **word count and suggestion**: e.g. “0/100 words (Suggested length: 50-100 words) .” As the user types, the count updates. If the user greatly exceeds say 150 words, the count might turn orange/red indicating it’s longer than typical. This isn’t a hard limit but a visual cue.

AI Features and Workflow: The Summary section is heavily enhanced by AI:

- **AI Summary Generator:** On clicking “Generate Summary”, the system will use available information (like the user’s job titles, years of experience, skills that have been entered so far) to craft a personalized summary. The generated summary is placed into the text area, and the preview updates to show it. We also display it with perhaps the first draft highlighted or with an “Undo” if they don’t like it. The user can edit the AI-generated text normally afterwards. This feature addresses writer’s block and can produce a strong starting point or even final copy in one click .
 - If the user hasn’t filled anything yet, they are navigated to the Work Experience section. There, a tooltip displays “Let’s start with your most recent work experience”. If the user hasn’t filled anything yet, they are navigated the user to the Work Experience section. There, a tooltip displays “Let’s start with your most recent work experience”.
- **Rephrase/Improve:** At any time, the user can highlight a sentence or the whole summary and click an “Improve” icon (magic wand). The AI will propose a clearer or more impactful version. This could be shown as an inline suggestion (highlighted text) that the user can accept or ignore. Alternatively, multiple rephrasing suggestions could appear in a small popup, letting the user choose their favorite. One-click “Improve All” will run the AI through the entire summary to polish grammar, eliminate passive voice, and add powerful keywords (especially if a job description is provided).
- **Keyword Suggestions:** If a target job is set, the AI checks whether key qualifications from the job posting are mentioned in the summary. If not, a suggestion might appear: e.g., “Consider mentioning **project management** in your summary.” The UI might underline the word “project management” in the suggestion color if it’s a recommended insertion. The user can click to accept (the AI will merge it into the text, making a minimal change like “... with 5+ years of experience in project management and team leadership.”). If multiple keywords are missing, the user can use “Incorporate All” to have the AI blend all of them in cohesively. This directly aligns with ATS optimization best

practices .

- **Autocomplete (Smart Compose):** While the user is typing the summary, if they pause, the AI might try to complete the current sentence. For example, the user types “Experienced software engineer skilled in”, the AI could gray-out suggest “ Java and Python development.” If the suggestion looks good, the user hits Tab to accept it. This speeds up writing by offering likely phrases. These suggestions use context (what’s already typed and common resume phrases) to be relevant.
- **Examples Library:** For additional inspiration, a link “View examples” could open a side panel with a few sample summaries from similar profiles (this could be powered by a library of resume examples as Resume.io had). For instance, if the user’s field is detected as Marketing, we show 2-3 example summary statements for marketing professionals. The user can copy one as a starting point or just glean ideas. This is another way to reduce the blank-page problem.

Validation: The summary text is free-form, so formal validation is minimal (no “required” enforcement). However, we do implement some checks:

- If the summary is extremely short (e.g., less than 10 words), we might show a tip: “A longer summary can make a stronger impression. Consider adding a couple of sentences.” This is more guidance than error.
- If it’s extremely long (like 300+ words), we might warn “Summaries are usually brief. Consider saving details for work experience.” Possibly even display a soft warning icon on the header for being overly long.
- We also ensure that any special characters or formatting the user might try (like using lists or excessive punctuation) are either stripped or warned if not supported by the template.
- Spelling/grammar: A spellcheck underlines misspelled words. Additionally, our AI can play a role in grammar checking as part of “Improve” – it can identify and fix third-person vs first-person issues or inconsistent tense.

Preview Rendering: As the user edits the summary, the preview shows the Summary heading (or an underline or however the template denotes it) followed by the summary text. The text flows as a normal paragraph. If the user hasn’t entered anything, no “Summary” label or text appears on the PDF. If the user renamed the section (e.g., “Profile”), the preview will show that new heading instead.

10× Better – Summary Section: Resume.io’s AI assistance for the summary was limited to a couple of tries at generation with no tailoring to specific jobs . OkBuddy’s summary editor is

vastly improved: unlimited AI-generated drafts, real-time keyword targeting, and smart completions. These tools let users create a compelling summary with minimal effort, turning a traditionally challenging part of resume writing into a quick, guided experience. For example, rather than manually writing and refining a paragraph, a user can generate a polished summary in one click and refine it by simply accepting AI suggestions, something not possible in older builders.

```
### Summary Section - Smart Guidance Logic

#### Empty State Detection & Guidance

**Smart Empty State Logic:**

```typescript
const hasExperience = cvData.experience.items.length > 0

const isEmpty = !summary.content.trim()

if (isEmpty && !hasExperience) {

 // Show guidance banner: "Bắt đầu dễ dàng hơn!"

 // Redirect to Work Experience section

}

...

Guidance Banner System:

- **Title:** "Bắt đầu dễ dàng hơn!"

- **Description:** "Hãy bắt đầu với kinh nghiệm làm việc để AI có thể hỗ trợ viết tóm tắt tốt hơn"

- **Action Button:** "Đi đến Kinh nghiệm làm việc"

- **Behavior:** Smooth scroll to Work Experience section
```

```
- Trigger: When Summary is empty AND no work experience exists
```

## Work Experience Section

**Purpose:** Document the user's employment history, typically the most important section. In the Dennis Schröder template, Work Experience entries are formatted with the job title, company, and location on the first line, the employment dates on the second line, and a bullet list of accomplishments below. This section can have multiple entries (jobs), each usually listed in reverse chronological order.

**Structure:** In the editor, the Work Experience section contains a list of **Job Entry sub-sections**. Each Job Entry has its own set of fields and perhaps its own sub-header. We represent each job as a card or sub-form within the Work Experience section, with a visual separation (like a border or spacing between jobs). Jobs can be reordered by dragging these sub-cards (using drag handles specific to each job). The fields for each job entry include:

- *Job Title* – text input (e.g., “Software Engineer”). Often shown in bold on the resume.
- *Company* – text input (e.g., “Google LLC”). Often shown next to or after the title on the resume.
- *Location* – text input (e.g., “Mountain View, CA”). Shown alongside company on resume first line. (This may also be optional; if the user doesn't want to list location, they can leave it blank.)
- *Start Date* – date picker or text (e.g., “Jan 2020”). For simplicity, we may use a month-year format. The field could be two dropdowns: Month and Year, or a combined control that forces that format.
- *End Date* – date or text (e.g., “Dec 2022” or “Present”). If currently employed, the user can toggle “Currently work here” which would disable End Date and instead output “Jan 2020 – Present”.
- *Description / Bullet Points* – This is a **repeating field list**, where the user can add multiple bullet points describing their responsibilities and achievements in that role. Each bullet could be an individual text field (single-line or expandable multi-line if it's long). We prefix them with a bullet “•” in the UI or simply style them so it's clear each entry will be a bullet. Next to each bullet field, there might be a small “–” drag handle or up/down arrows to reorder bullets, and a trash icon to delete that bullet. The user can click “Add bullet” to append a new empty bullet field. By default, perhaps 2 bullet fields are provided empty to start with, to cue the user that they can list achievements.



Additionally, each Job Entry sub-card has its own mini-header displaying the Job Title and Company for quick identification (especially when collapsed or reordering). We might allow collapsing individual job entries too: e.g., a caret next to the job title could collapse the bullet list for that job, showing only the title/company and dates – this is useful when a user has many jobs; they might want to collapse older ones while focusing on the current one. When collapsed, the sub-card could shrink to one or two lines.

### **Default/Empty State:**

- If the user has no work experience added yet, the section will initially be empty except for perhaps an “Add Work Experience” button. The empty state message might say: “No work experience added yet. Click ‘Add Work Experience’ to include your first job.” When clicked, a blank Job Entry form appears with fields for title, company, etc. We ensure at least one entry form is present when the user starts entering data. Resume.io typically starts with one dummy entry or at least an obvious add button.
- Once one job is added, the “Add Work Experience” button remains at the bottom of the section to add more jobs.
- Each new job entry may come with some placeholder text in fields to guide (e.g., Job Title placeholder “e.g. Sales Manager”, Company placeholder “e.g. Acme Corp”). Bullet fields might have greyed example text like “e.g. Managed a team of 5 sales associates...” to hint at the type of content.

### **Data Entry & Validation:**

- **Job Title/Company:** We recommend at least one of these be filled for an entry to be considered valid. If the user leaves the job title blank but fills company, or vice versa, we won’t forbid it (some might put “Intern” as title with company, or no company if it’s freelance?), but we’ll mark incomplete entries in a subtle way. Ideally both should be filled. We can highlight if either is missing: e.g., a red outline if title is empty when company is filled or vice versa, with tooltip “Job title missing”.
- **Dates:** When the user picks dates, enforce logical consistency. If an end date is earlier than start date, we show a validation error on the End Date field: “End date must be after start date.” If the user checks “Present” for current job, ensure End Date is disabled and shown as “Present” in preview. If they then fill an End Date, auto-uncheck Present. For date input, we might not allow a future date beyond the current month (except Present). Also, date fields could be optional (some might just put years). If a date is left blank, we handle gracefully (e.g., show just one date if needed or nothing).
- **Bullet Points:** There’s no strict validation on content of bullets, but we might implement:

- If a bullet field is left completely empty (and it's not the only bullet), we can auto-remove it on save/exit, or prevent creating a new bullet until the current one has text (so we don't accumulate empties).
- Perhaps enforce a character limit per bullet for readability (e.g., if a bullet > 300 characters, we might suggest breaking it into two bullets).
- If no bullets are provided for a job, maybe the user isn't done – we should encourage at least one bullet. We can highlight “Consider adding a description of your role.” However, if they truly want to leave it blank (rare), it would just show title/company and dates on resume with no description.
- **Entry Deletion:** Each job entry may have a delete button (trash icon) to remove that job entirely. On clicking, confirm because that will wipe out potentially multiple bullet texts. If confirmed, remove the entry card and update preview. This is useful if the user added an entry by mistake or decides to omit a job.

**AI Enhancements:** This is a critical area (if not the most, with Summary Section close behind) where AI can assist and drastically cut down typing:

- **Job Description Autocomplete:** When the user starts adding a job, after they fill in *Job Title* (and maybe Company), the system can fetch context-specific content. For example, suppose the user enters “Marketing Manager” as title. The editor can automatically fetch a set of **sample bullet points** for a Marketing Manager role. These come from the OkBuddy Smart Internal Library described below. A prompt can appear: *“Need help describing your role? Here are 3 example responsibilities for a Marketing Manager:”* followed by bullet suggestions like:
  - “Developed and executed marketing campaigns that increased lead generation by 40%.”
  - “Managed a team of 5 marketing specialists, providing guidance and performance feedback.”
  - “Analyzed market trends and customer data to inform product positioning and strategy.”
- Each suggestion might have an “ Add” button next to it. Clicking “Add” will insert that text into a new bullet field in the editor. The user can then tweak numbers or specifics as needed. They can add multiple suggestions or ignore ones that don't fit. This **example-based starter** approach provides immediate content to work with, addressing the pain point of writing job descriptions from scratch .

- **AI Bullet Generator:** Alternatively (or in addition), the user might click an “Auto-generate bullets” button which uses GPT to generate, say, three bullet points based on the job title and perhaps a brief prompt about the role. This could even consider the industry if the Company field is a known entity (e.g., if company is Google, maybe the AI knows relevant context to mention scale or tech). The generated bullets appear either directly in the fields (highlighted until confirmed) or in a preview list for the user to accept. This GPT pre-fill can produce a quick first draft of the entire job description.
- **Bullet Sentence Builder:** For users who want more control, we can offer a guided bullet composer. For example, when adding a new bullet, instead of a blank text field, it could show something like:
  - [Select Action Verb] + [What you did] + [Result].

The user could choose from a dropdown of action verbs (lead, managed, developed, implemented, etc.), then type the specifics, then possibly add a result. The AI can populate these lists or suggest completions. This ensures strong phrasing and encourages including results. For instance, the user picks “Implemented”, types “a new CRM system”, then the AI suggests for result “, resulting in a 20% increase in sales productivity.” The user can accept that suggestion. This “mad-libs” style construction is optional (user can always just type directly), but it provides a scaffold to create effective bullet points with metrics.
- **AI Metric Booster:** A common feedback for resumes is to include quantitative results. Our AI will analyze each bullet point for presence of metrics or specific outcomes. If a bullet is purely task-oriented (“Managed weekly team meetings.”), the AI might recommend elaboration: “Consider adding a result or metric to this bullet.” This could be delivered as a subtle suggestion icon next to the bullet or as part of an “Improve” action. If the user clicks it, the AI could ask a follow-up in a small dialog: *“Do you have a measurable result for this task? e.g., time saved, revenue increased, etc.”* If the user provides something (say “improved team productivity by 15%”), the AI will append or integrate that: “Managed weekly team meetings to streamline project updates, **boosting team productivity by 15%.**” If the user doesn’t have one, the AI might suggest a generic improvement or leave it. We want to gently push users toward achievement-focused bullets.
- **Bullet Rewriting and Optimization:** Each bullet can have a one-click “**Improve Bullet**” option (like a magic wand icon at the end of the field). Pressing it sends that single bullet to the AI for rewriting in a more impactful way (active voice, stronger verbs, concise phrasing). The result replaces the text if accepted. The user can repeatedly click to cycle through alternate phrasings if desired. Similarly, a “**Rephrase All**” for the job can run through every bullet and title (except it won’t change the actual job title or company) to uniformly enhance wording. For example, if the user wrote a bullet “Responsible for

managing project timelines”, the AI might rewrite to “Managed project timelines to ensure on-time delivery.” – small but meaningful improvements.

- **Keyword Tailoring:** With a job description loaded, the AI will check each job entry against relevant keywords. For example, if the target listing mentions “Agile methodologies” and one of the jobs is in software, the editor might highlight that none of the bullets mention “Agile”. It could then suggest: “Mention Agile methodologies in this experience.” This might appear as a highlighted suggestion. The user could then either manually incorporate it or let AI do it: clicking the suggestion could generate a bullet “Integrated Agile Scrum practices to improve team workflow.” The user could accept and adjust if needed. This ensures each job entry maximizes its relevance to the target role .
- **Apply All Suggestions:** Within the Work Experience section, if multiple improvements are detected across bullets (say the AI has suggestions for 3 bullets), the user can click an “Apply All” at the section header level to accept every AI suggestion in one swoop. For example, if 2 bullets had minor grammar fixes and 1 had a missing keyword suggestion, one click would fix all three issues at once. This saves time reviewing each suggestion individually.
- **Auto-ordering & Chronology Help:** Perhaps a minor feature – if the user enters job dates that are out of order, we could prompt “Sort by date?” offering to automatically reorder the job entries by date (most recent first). The user can accept, which triggers a reordering (same as drag-and-drop but automated). This is a convenience to maintain chronology.

```
Experience Section - Core Flow Logic
```

```
Empty State Guidance System
```

```
Blue Banner Display Logic:
```

```
- **Trigger Condition:** Both Position and Company fields are empty
```

```
- **Banner Content:**
```

```
- **Title:** "Xây Dựng Kinh Nghiệm Làm Việc Ấn Tượng Trong 5 Giây!"
```

```
- **Description:** "Chỉ cần nhập chức danh và tên công ty - OkBuddy AI giúp bạn tạo mô tả kinh nghiệm làm việc ngay lập tức."
```

```
- **Persistence Logic:** Banner remains visible until BOTH fields contain text
```

- **Behavior:** Banner disappears only when user has finished typing in both Position AND Company fields

#### AI Button State Management

**Critical UX Pattern - Empty vs Content States:**

**🔵 When Field is EMPTY:**

- **Main CTA (Header):** "Tạo mới" button
  - `variant="primary"` (blue/prominent)
  - `size="md"` (larger)
  - Active and clickable
  - Highly visible and dominant
- **Sub CTA (Bottom):** "Cải thiện với OkBuddy AI" button
  - Grayed out and disabled
  - `cursor="not-allowed"`
  - Shows what will be available after content is created

**✅ When Field has CONTENT:**

- **Header:** "Tạo mới" button becomes
  - Small, outline style
  - Disabled state
  - Less prominent
- **Bottom:** "Cải thiện với OkBuddy AI" button becomes

- Main prominent CTA
- `variant="primary"` (blue/prominent)
- Fully active and clickable

#### **Perfect User Flow:**

1. **Empty state** → User's attention drawn to prominent blue "Tạo mới" button in header
2. **With content** → User's attention drawn to prominent "Cải thiện" button at bottom

#### #### Disabled Button Styling Standards

##### **Primary Variant (Disabled):**

- **Background:** `bg-gray-200` (light gray, not dark gray)
- **Text:** `text-gray-400` (muted gray)
- **Border:** None

##### **Outline Variant (Disabled):**

- **Background:** `bg-gray-100` (light background)
- **Border:** `border-gray-200` (lighter border)
- **Text:** `text-gray-400` (muted gray)

**Consistency Rule:** All disabled AI buttons maintain the same light gray, muted appearance across both primary and outline variants.

**Reordering & UI Behavior:** As mentioned, jobs can be dragged to reorder. The section might automatically sort new entries by date by default, but manual override is allowed. Dragging is done via a handle on the job's mini-header (maybe next to the job title). When dragging a job, its whole card moves; the other job cards shift. This is done within the Work Experience section's container. If a job entry is expanded, it remains expanded after moving. If it's collapsed, it drags as a small bar. We ensure the drag-and-drop is smooth and doesn't interfere with dragging the whole section (there's distinct handles for section vs entry).

**Preview Rendering:** Each job entry on the preview will look like:

- **Line 1:** Job Title – Company, Location (the title and company are separated by an en-dash or something, with location after a comma, as per the template style). We might style the Job Title bold and company regular, etc., but that's template-specific. The user's renaming of the section to "Experience" doesn't affect individual entries, just the heading.
- **Line 2:** Start Date – End Date (italic or smaller font perhaps). If currently employed, End Date is "Present". If only year given, we just show year or month-year based on input.
- **Next Lines:** Each bullet as a bullet point (•) indented under that job. The formatting includes proper indentation and spacing consistent in the template. If a job has no bullets (not recommended, but possible), it might just show the two lines of title and date and then directly the next entry's heading.
- Entries are separated by a small vertical gap for clarity.
- If the user reorders entries or edits text, these lines update live. Long bullets will wrap as needed in the preview's width.

OkBuddy's Work Experience editor dramatically reduces the effort needed to document each job, far surpassing Resume.io's static examples approach:

- **Firstly**, the integration of AI suggestions means users can populate an entire job description with one or two clicks, pulling from a rich library of sample phrases or direct AI generation.
- **Secondly**, every bullet point can be optimized in place – no need to write perfectly from scratch when the AI will polish it or suggest improvements. **Resume.io provided example sentences to copy manually , whereas OkBuddy inserts them directly and adapts them to your context.** The AI metric booster ensures users highlight their impact with numbers, a nuance typically overlooked without expert advice. And tailoring content to a job posting is straightforward; the user is actively guided to include exactly what recruiters are looking for (e.g., if a job requires "CRM software", the AI might prompt adding a line about CRM experience in one of the jobs).

- All of this yields a richer, keyword-optimized Work Experience section with a fraction of the manual typing. Users no longer have to labor over phrasing or worry if they included the right details – the AI co-pilot has their back.

## Skills Section

**Purpose:** To list the user’s key skills or areas of expertise, often as bullet points or tags. In the Dennis Schröder CV template, the Skills might be presented as a simple list or a series of comma-separated values. There might not be detailed descriptions here, just keywords indicating competencies (e.g., “Project Management, Python, Data Analysis, Leadership”). This section gives a quick overview of what the candidate is good at.

**Field Type:** A **tag input** or **chip list** interface. The user can add multiple skills, one by one. Typically:

- There is an input box that prompts “Type a skill and press Enter”.
- When the user presses Enter (or a comma), the text is converted into a **skill chip** (a UI element that shows the skill text, possibly with a border or background).
- Each chip has a small “x” to remove it if needed.
- The input then clears for the next skill.
- Alternatively, we can have a multi-line textarea where each skill is separated by commas or new lines, but the chip approach is more modern and allows suggestions as you type.

**Empty State:** If no skills are added yet, the section might show something like: “*Add your skills – e.g. Project Management, SEO, Data Analysis*” as placeholder text in the input field. Additionally, if a target job or previous sections give clues, the editor might pre-populate a **suggestions list** of potential skills to add. For instance, if the user’s Work Experience bullets frequently mention “budgeting” or “Excel”, the skills section empty state could suggest those. Or if the job description is loaded, and it requires certain skills the user hasn’t added, those appear as chips in a “Suggestions:” area that the user can click to add. For example, *Suggestions: [Python] [+], [Machine Learning] [+], [Public Speaking] [+]* might show if relevant. Clicking the “+” on a suggestion adds it as a confirmed skill chip.

**Adding Skills:** The user types a skill name. As they type, an autocomplete dropdown appears with matching skill suggestions. This serves both as a shortcut and a standardization (for instance, typing “Photos” might suggest “Adobe Photoshop”). The suggestion database can be broad (common skills library) and also contextual:

- If the user’s field (from summary or work entries) is engineering, suggest technical skills like programming languages.



- If a target job description is provided, prioritize suggestions that appear in the job requirements (e.g., if “SQL” is in the JD, as soon as the user types “S”, suggest “SQL”).
- If the user has a skill very similar to one already listed, we might warn to avoid duplicates or unify them (e.g., they have “Excel” and then add “Microsoft Excel” – probably the same skill).

The user can either choose a suggestion or finish typing a custom skill and press Enter.

Each added skill becomes a chip. The chips are usually displayed inline, wrapping to the next line if they exceed the width. They might appear sorted in the order entered, or alphabetically if we decide to auto-sort (but manual order might allow the user to highlight more important skills first, so we’ll keep manual ordering).

**Reordering Skills:** The user can reorder skill chips if desired. This could be done by dragging a chip (if the UI library supports drag of chips) or by remove and re-add. We can make chips draggable for convenience: clicking and dragging a chip will move it in the list. A visual placeholder (like a ghost chip) shows the new position. This helps if the user wants their most relevant skills first. If not trivial to implement, we might skip explicit drag for chips and rely on order-of-entry.

**Deletion:** Clicking the “x” on a chip removes that skill. There’s typically no confirmation (since adding it back is easy). If the last skill is removed, we’re back to empty state UI.

**Skill Proficiency (Optional):** Resume.io had an option to toggle skill level bars . In our spec, by default, Dennis’s template might not include skill proficiency indicators (the template description didn’t mention it). We will keep it simple: just a list of skills. However, we note that if desired, a proficiency can be added by the user – e.g., after typing a skill, they could select a level (like 1-5 or beginner/intermediate/expert). The UI could have a tiny dropdown on each chip or a star rating. If enabled, the preview might show a small graphical indicator next to that skill. We mark this feature as optional/configurable in template settings. The default for this template is off (no bars).

**Validation:** There’s little to validate for skills as they are short text:

- We avoid empty string as a skill (the input requires some non-space characters to accept).
- If a duplicate is entered (case-insensitive match to an existing skill), we can ignore it or show a brief tooltip “Skill already added.” No duplicates means the user’s list stays unique.
- If a skill name is extremely long (say the user pastes a whole sentence), we might trim it or warn (“Skills should be short, consider using the Summary for full sentences”).

- We might restrict each skill to, for example, 2-3 words max (commonly skills are single words or a phrase like “Project Management”).

**AI and Suggestions:** The Skills section benefits from AI mainly in suggesting what to include:

- **Job-based Suggestions:** If a job description is provided, our system will parse out the required or desired skills from it. These could be technical skills, soft skills, certifications, etc. The editor will then surface these as suggestions in the skills section. For example, if the job requires “Python, SQL, and project management,” and the user hasn’t added those, they will appear in a highlighted suggestion list as mentioned. The user can add all with one click (“Add all suggested skills”) or individually. This ensures the resume’s skills list aligns closely with the target job’s keywords – a crucial factor for ATS filtering.
- **Resume-based Suggestions:** The AI looks at the content of Work Experience and Summary to infer skills that the user implicitly has but might not have explicitly listed. For instance, if a bullet says “Led marketing campaigns on Facebook and Google,” the AI might suggest adding “Digital Marketing” or “Social Media Marketing” as skills. These appear as suggestions like “Consider adding: Digital Marketing”. Similarly, if the Summary mentions a strength (“strong communicator”), maybe suggest “Communication” skill.
- **Industry Suggestions:** If the user has specified an industry or role, and their skills list looks sparse, we can proactively suggest common skills for that role. E.g., “As a Data Analyst, you might include: [Data Visualization] [SQL] [Critical Thinking]”.
- **Skill Validation (Relevance Checker):** Conversely, the AI can identify if any listed skill might be irrelevant or outdated for the target role. If the user added something that the job description or modern standards don’t value (e.g., listing “Microsoft Word” for a software developer position), the system might gently flag that as low-impact. Possibly an icon on the chip or a note like “Low relevance to target job” so the user can consider removing it to save space. This keeps the skills section focused and effective.
- **One-Click Add All:** If multiple suggestions are present, a button can allow adding all suggested skills at once. The AI will automatically populate the skill chips. The user can then remove any they don’t actually have or want to display. We trust the user to only keep truthful skills, but we facilitate the discovery.
- **Ordering Suggestion:** The system might automatically sort added skills by relevance if a job is targeted – e.g., skills that appear in the job posting or are highly sought can be highlighted first. If implemented, we’d notify the user like “Skills have been ordered by relevance. You can reorder if you prefer.”
- **No heavy writing required:** Because skills are just labels, we don’t need an AI to “write” anything here, but the intelligence lies in ensuring the right skills are present. This

drastically helps users who might not realize which keywords to list. It's essentially an **ATS keyword optimizer for skills**. Resume.io had a simple tool suggesting relevant skills which the user could click to add – OkBuddy amplifies this by customizing suggestions to the actual job target and the user's unique background.

**Preview Rendering:** In the final resume PDF, skills will be presented likely as a comma-separated list or bullet list under a "Skills" heading. For example:

**Skills:** Leadership; Project Management; Python; SQL; Data Analysis; Communication.

The exact format (bullets vs commas vs inline tags) depends on the template's style. Dennis's template might have them centered or left, but likely a simple list. If skill level bars were enabled, each skill might have a tiny bar or rating next to it on the preview. By default, we assume no bar, just text. The preview updates as each skill is added or removed. If the section is empty, no "Skills" heading appears.

**Skills Section:** OkBuddy's skills editor ensures users don't overlook critical skills and includes them with minimal effort. Resume.io offered a basic list of related skills to pick from, but **OkBuddy takes it further:** it cross-references the desired job and the user's own content to provide a tailor-made set of skill suggestions. The "Relevance Checker" powered by AI means every skill on the resume serves a purpose; users are nudged to include in-demand skills and omit trivial ones. This targeted approach, combined with one-click adding and auto-complete, makes building a powerful skills list virtually effortless. The user can essentially curate a perfect skills profile in seconds, something that would normally require researching job ads or using external tools. This level of integration is a significant leap over standard resume builders.

## Education Section

**Purpose:** List the user's academic background, such as degrees and institutions attended, along with dates of study and possibly honors or relevant details. The Dennis Schröder template likely shows Education similarly to Work Experience, but usually shorter. Typically, the degree and field of study, institution name, location, and graduation date are included.

**Structure:** Like Work Experience, the Education section may contain multiple **Education Entry** sub-sections (for multiple degrees). Each entry includes fields:

- *Degree & Field* – text input (e.g., "B.Sc. in Computer Science"). The user might separate degree and field into two fields (Degree type and Major), but a single combined field might be simpler. Alternatively:
  - Degree (e.g. "Bachelor of Science") – could be a dropdown or text.
  - Field of Study (e.g. "Computer Science") – text.

- *Institution Name* – text input (e.g., “Stanford University”).
- *Location* – text input, optional (e.g., “Stanford, CA”). Many resumes include location of the school, but it could be omitted.
- *Graduation Date* (or Date Range) – often just the graduation year or year and month. Could be a Year field (and Month if we want). Or Start and End year if it was a multi-year program (less critical to list start for education, usually just graduation year or expected grad).
- *Additional Details* – optional multi-line field for things like GPA, honors, relevant coursework, thesis title, etc. This is not always used, but we can allow a small text area if the user wants to include something like “Graduated with Honors (GPA 3.8)” or “Relevant coursework: Data Structures, Algorithms”.

If multiple entries exist, they are typically listed with most recent degree first (especially if one is current, like a student listing education might list current program first).

**Empty State:** If no education is added yet, similar to work, we show “No education added. Click ‘Add Education’ to list your academic credentials.” One blank entry may be provided by default or appear when the user clicks Add. If the user is a current student or very early career, sometimes one education is what they start with. We could pre-add one entry form to prompt them to fill it.

Placeholders guide the input:

- Degree: maybe a dropdown with common degrees (High School Diploma, B.A., B.Sc., M.A., M.Sc., Ph.D., etc.) to choose from, to standardize format. Or placeholder “Degree (e.g. Bachelor of Science)”.
- Field: placeholder “Field of study (e.g. Computer Science)”.
- Institution: “Institution name (e.g. University of XYZ)”.
- Dates: if using dropdowns, default could be blank like “Start Year – End Year”. If using a single Graduation Year field, placeholder “Year (e.g. 2024)”.
- Additional details: placeholder “e.g. Graduated magna cum laude; GPA 3.9” or simply left empty.

### Input Behavior & Validation:

- None of these fields are absolutely required, but typically at least *something* should be present per entry for it to make sense. We will require an Institution or Degree at

minimum. If both are blank, that entry is effectively empty and we won't show it on preview.

- If the user enters only a degree and no institution, we might warn "Institution name missing." If only institution and no degree, that might be okay for something like short courses, but generally, we expect a degree.
- If a date is provided, ensure it's a valid year or year-month. If Start and End are both used, ensure Start is not after End. If only End (grad year) is used, fine. If currently studying, perhaps allow "Expected [Year]" or a "Present" toggle if they haven't graduated yet. For example, a current student might put "2023 – Present" or "Expected 2025". We might offer a checkbox "Currently studying" similar to work experience's present, which will output "Expected [Year]" or simply "Present".
- Location is optional, no strict validation, but similar hints as before (if filled, ensure it's not overly long).
- Additional details: optional, no validation beyond length. If a user tries to list multiple items, maybe encourage using semicolons or separate lines – the preview will likely show them as a continuation under the degree line in smaller text.

**Reordering:** If multiple degrees are entered, the user can reorder them by dragging their sub-cards (like jobs). We might initially keep them sorted by date (most recent first), but allow manual moves. Drag handle on each education entry's top (maybe near the degree name). Preview updates accordingly.

**AI Assistance:** Education is factual, so AI has a smaller role, but we can still enhance user experience:

- **Auto-complete Institution Names:** As the user types an institution, we can suggest full official names. E.g., typing "MIT" suggests "Massachusetts Institute of Technology". This ensures consistency and correct spelling. Similarly for high schools or certifications. We might use a list of known universities worldwide to power this.
- **Standardize Degree Format:** If the user types "bsc computer science", the system can auto-correct casing and format (e.g., "B.Sc. in Computer Science"). It could even detect synonyms – if they type "Bachelor of Arts in Psych", finalize as "B.A. in Psychology" properly. This could be either through predefined rules or an AI model that recognizes education credentials and formats them nicely.
- **Achievements or Coursework Suggestions:** If the user does use the additional details field (or we detect they might need content there, like for a current student with no work experience, they might want to list coursework or projects), AI can prompt something like: "Mention relevant coursework or honors to strengthen this section." Possibly offer

examples: e.g., for a Computer Science student, suggest “Relevant coursework: Algorithms, Data Structures, Machine Learning” or “Capstone Project: Developed a mobile app for...”. The user can accept an example and then personalize it.

- **Job Target integration:** If the target job requires a certain degree or certification and the user hasn’t listed it, that’s critical to highlight. For instance, if job description says “MBA preferred” and the user doesn’t have an MBA in education, obviously the AI can’t fabricate one – but it could prompt the user if maybe they have an equivalent or partial (“Do you have an MBA or currently pursuing one? If yes, add it to Education.”). This might be rare and more of a user decision, so perhaps just noting if an expected education credential is missing (like job requires “Bachelor’s in Finance” and user’s highest is high school, the system could flag that in recommendations panel).
- If the user has multiple entries and is unsure how to prioritize, AI might suggest ordering by relevance (e.g., if they have a Ph.D. and a B.Sc., obviously Ph.D. first – we can auto-sort by level or year).
- **No heavy AI writing** here except maybe formatting and tiny suggestions, since education content is straightforward.

**Preview Rendering:** For each Education entry, likely:

- **Line 1:** Degree/Field – Institution, Location (similar style to work exp). Or possibly Institution – Location on one line and Degree on the next; but given the prompt, they specifically only spelled out the work experience format. Common resume practice is Degree & Field, then institution. We will format as:
  - e.g., *Bachelor of Science in Computer Science – Stanford University, Stanford, CA* (all on one line). If that’s too long, template might break it into two lines, but let’s assume one line for now.
- **Line 2:** Dates (e.g., *2016 – 2020*). If currently ongoing, maybe *2019 – Present* or *Expected 2025*. If only one year (graduation year), we might just show that single year or “Class of 2020”.
- **Optional line 3:** Additional details like GPA or honors if provided. These might be in parentheses or smaller font. Example: *Graduated **magna cum laude**, GPA 3.8/4.0*. If multiple items (coursework, etc.), possibly separated by commas on that line.
- Each entry is separated by a small gap. No explicit bullets since these are not bullet lists.
- Section heading “Education” is shown (or whatever it’s renamed to).

- If section is empty, no heading shown.

**If the user renames the section** (e.g., to “Academic Background”), that heading updates accordingly in preview.

**10× Better – Education Section:** While education is mostly straightforward data entry, OkBuddy still introduces small but meaningful improvements. The auto-complete for institutions and degree formatting saves time and ensures correctness – something not typically found in basic resume builders. The AI’s subtle suggestions (like adding coursework for students or honors for graduates) help users maximize this section’s impact. Resume.io didn’t focus AI here (no need to), but OkBuddy’s attention to detail (like formatting “B.Sc.” properly) gives a polished result effortlessly. Moreover, integrating job description awareness (e.g., prompting a user to add a certification if the job expects it) is an extra layer of smart guidance that makes the final resume more aligned with job requirements. These little enhancements contribute to the overall 10× improved experience, ensuring even the Education section isn’t an afterthought.

## Other Custom Sections

OkBuddy allows users to add any number of custom sections beyond the defaults. These can represent categories like **Certifications, Projects, Volunteer Experience, Publications, Languages**, etc. When a user adds a custom section, they choose a template for it:

- A **List Section** template, which behaves like Work Experience/Education (multiple entries, each with a title and description, possibly dates). Good for things like Projects or Volunteer Experience where you might list multiple items with descriptions.
- A **Simple Text Section** template, which is just a single text area (like Summary) for a paragraph of text. Could be used for an “About Me” or similar section.
- A **Tag List Section** template, like Skills, useful for something like Languages (where you list multiple languages, possibly with proficiency) or Hobbies.

The user can then fill these similarly to the patterns we’ve detailed:

- If it’s a list, they’ll have “Add [item]” buttons and AI help to generate content if relevant.
- If it’s a tag list, they can add items with suggestions (for example, in a Languages section, typing a language might suggest “English – Native, French – Intermediate” as structured entries if we had proficiency).

```
Custom Section - Product Specifications
```

## ## 🌐 Overview

The **Custom Section** feature allows users to add flexible, user-defined sections beyond the standard CV sections (Contact, Summary, Experience, Education, Skills). This feature provides maximum customization while maintaining the professional structure and AI-powered assistance of the OkBuddy platform.

---

## ## 🏗️ Architecture & Data Structure

### ### CVData Integration

**Dynamic Section Support:**

```typescript

```
interface CVData {  
  
  // Core sections (non-deletable)  
  
  contact: ContactData  
  
  summary: { content: string; usedAI: boolean }  
  
  experience: { items: ExperienceItem[]; usedAI: boolean }  
  
  education: { items: EducationItem[]; usedAI: boolean }  
  
  skills: { items: SkillItem[]; usedAI: boolean }  
  
  // Dynamic custom sections with pattern: [sectionId: string]: any  
  
  [customSectionId: string]: CustomSectionData  
  
}
```



```
interface CustomSectionData {  
  
    type: 'custom'  
  
    title: string  
  
    items: CustomSectionItem[]  
  
    usedAI: boolean // Always false for custom sections  
  
}
```

```
interface CustomSectionItem {  
  
    id: string  
  
    title: string          // "Tên hoạt động, giải thưởng"  
  
    organization: string   // "Tổ chức"  
  
    location: string       // "Địa điểm"  
  
    startDate: string  
  
    endDate: string  
  
    current: boolean  
  
    bullets: string[]  
  
}
```

...

****Section ID Pattern:****

- Custom sections use unique identifiers: `custom-\${timestamp}` or `custom-\${uuid}`
- Example: `custom-1703123456789`, `custom-projects`, `custom-certifications`
- Allows unlimited custom sections with unique identification

🍷 User Interface Design

Add Custom Section Interface

****Add Button Location & Styling:****

- ****Position:**** Bottom of the sections list in CV Editor panel
- ****Button Text:**** "Thêm mục tùy chỉnh"
- ****Visual Design:****
 - Full-width button with dashed border
 - ``border-dashed border-gray-300``
 - Blue text color: ``text-blue-600``
 - Hover state: ``hover:bg-blue-50``
 - Plus icon: ``<PlusIcon size={16} className="mr-1" />``

****Button Behavior:****

```typescript

```
const handleAddCustomSection = () => {

 const newSectionId = `custom-${Date.now()}`

 const newSection: CustomSectionData = {

 type: 'custom',

 title: 'Mục tùy chỉnh', // Default title, user can edit
```

```

 items: [],

 usedAI: false
 }

 updateCVData({
 ...cvData,

 [newSectionId]: newSection
 })
}

...

Custom Section Header

Section Header Components:

1. **Editable Title:** Default "Mục tùy chỉnh" - user can click to edit
2. **Drag Handle:** 6-dot grip for reordering (same as core sections)
3. **Collapse/Expand Toggle:** Chevron icon for content visibility
4. **Delete Button:** Trash icon - ONLY available for custom sections
5. No AI Indicator: Custom sections don't show AI suggestion indicators

Delete Confirmation Flow:

```typescript
const handleDeleteCustomSection = (sectionId: string) => {

  // Show confirmation modal

```

```

const confirmed = confirm(`Bạn có chắc chắn muốn xóa mục "${sectionTitle}"? Hành động này không thể hoàn tác.`)

if (confirmed) {

  const updatedData = { ...cvData }

  delete updatedData[sectionId]

  updateCVData(updatedData)

}

}

...

---

## 📝 Field Structure & Labels

### Vietnamese Field Labels

**Customized Field Mapping:**

- **Title Field:** "Tên hoạt động, giải thưởng"

  - Placeholder: "VD: Giải nhất cuộc thi, Hoạt động tình nguyện..."

  - Use case: Awards, volunteer work, projects, certifications

- **Organization Field:** "Tổ chức"

  - Placeholder: "VD: Đại học XYZ, Công ty ABC..."

  - Use case: Institution, company, organization name

- **Location Field:** "Địa điểm"

```

- Placeholder: "VD: Hà Nội, TP.HCM..."
- Use case: Geographic location of activity
- ****Date Fields:**** Standard start/end date with "Present" option
- Start Date: "Ngày bắt đầu"
- End Date: "Ngày kết thúc"
- Current checkbox: "Đang tiến hành"

Field Behavior & Validation

****All Fields Optional:****

- No required field validation for custom sections
- Users can leave any field blank without errors
- Empty fields simply don't appear in the preview
- Flexibility for various use cases (some activities may not have locations, dates, etc.)

****Date Handling:****

```typescript

```
const handleDateUpdate = (itemIndex: number, field: 'startDate' | 'endDate', value: string) => {
```

```
 const updatedItems = [...sectionData.items]
```

```
 updatedItems[itemIndex] = {
```

```
 ...updatedItems[itemIndex],
```

```
 [field]: value
```

```
 }
```

```
// No strict validation for custom sections

updateSection({

 ...sectionData,

 items: updatedItems

})

}

...

🛠️ Functional Requirements

Item Management

Add New Item:

- Button text: "Thêm mục"

- Creates empty item with default structure

- No AI pre-population (unlike Work Experience)

- User fills manually or leaves fields as needed

Remove Item:

- Each item has individual delete button

- Confirmation: "Bạn có chắc chắn muốn xóa mục này?"

- Immediate removal without undo
```

### **\*\*Reorder Items:\*\***

- Drag & drop functionality same as Work Experience
- Items can be reordered within the custom section
- Maintains chronological flexibility

### **### Bullet Point System**

#### **\*\*Description Bullets:\*\***

- Label: "Mô tả chi tiết"
- Multiple bullet points supported
- Each bullet can be deleted individually
- "Thêm mô tả" button to add new bullets
- No AI assistance for bullet generation

#### **\*\*Bullet Management:\*\***

```typescript

```
const handleUpdateBullet = (itemIndex: number, bulletIndex: number, value: string) =>
{
  const updatedItems = [...sectionData.items]

  updatedItems[itemIndex].bullets[bulletIndex] = value

  updateSection({ ...sectionData, items: updatedItems })
}
```

```
const handleAddBullet = (itemIndex: number) => {

  const updatedItems = [...sectionData.items]

  updatedItems[itemIndex].bullets.push('')

  updateSection({ ...sectionData, items: updatedItems })

}
```

...

🎯 Use Cases & Examples

Common Custom Section Types

****1. Projects & Portfolio:****

...

Title: "Mobile App Development"

Organization: "Personal Project"

Location: "Remote"

Dates: "Jan 2024 - Mar 2024"

Bullets:

- "Developed iOS app using Swift and CoreData"
- "Implemented real-time messaging with Firebase"
- "Published to App Store with 4.8★ rating"

...

****2. Certifications & Training:****

```

Title: "AWS Solutions Architect - Associate"

Organization: "Amazon Web Services"

Location: ""

Dates: "Dec 2023"

Bullets:

- "Certification ID: AWS-ASA-123456"
- "Valid until December 2026"

```

****3. Volunteer Experience:****

```

Title: "Environmental Conservation Volunteer"

Organization: "Green Earth Vietnam"

Location: "Ho Chi Minh City"

Dates: "Jun 2023 - Present"

Bullets:

- "Organized beach cleanup events for 200+ participants"
- "Led educational workshops on sustainable living"

```

****4. Awards & Recognition:****

```
'''
Title: "Employee of the Year"
```

```
Organization: "Tech Corp Vietnam"
```

```
Location: "Ho Chi Minh City"
```

```
Dates: "Dec 2023"
```

```
Bullets:
```

- "Recognized for outstanding performance and leadership"
- "Awarded for increasing team productivity by 30%"

```
'''
```

```
---
```

```
## 🚫 AI Integration Limitations
```

```
### No AI Features
```

```
**Deliberate AI Exclusion:**
```

- Custom sections do NOT have AI generation capabilities
- No "Generate" or "Improve" buttons
- No job description targeting suggestions
- No Smart Internal Library integration
- `usedAI` field always remains `false`

```
**Rationale:**
```

- Custom sections are highly specific to individual circumstances
- AI training data may not cover specialized activities
- User creativity and manual input preferred for unique experiences
- Maintains authenticity and personal touch

Manual Content Only

****User Responsibility:****

- All content must be manually entered
- No autocomplete or suggestion features
- Users rely on placeholder text and examples for guidance
- Encourages thoughtful, personalized content creation

Preview Rendering

CV Preview Display

****Section Rendering:****

- Custom sections appear in the CV preview with user-defined titles
- Empty custom sections are hidden from preview (same as core sections)
- Items formatted similarly to Work Experience entries
- Maintains Dennis Schröder template styling consistency

****Format Example:****

```

## PROJECTS & PORTFOLIO

Mobile App Development - Personal Project, Remote

Jan 2024 - Mar 2024

- Developed iOS app using Swift and CoreData
- Implemented real-time messaging with Firebase
- Published to App Store with 4.8★ rating

```

****Styling Consistency:****

- Section headings: Same font and style as core sections
- Item formatting: Title-Organization-Location pattern
- Date display: Standard month/year format
- Bullet points: Consistent indentation and spacing

🔄 Section Management

Core vs Custom Section Distinction

****Non-Deletable Core Sections:****

- Contact Information
- Summary/Objective
- Work Experience
- Education
- Skills

****Deletable Custom Sections:****

- Any user-created section
- Projects, Certifications, Volunteer Work, etc.
- Complete removal from CVData structure

DraggableSection Integration

****Section Header Logic:****

````typescript`

```
const DraggableSection = ({ section, sectionId, onDelete, onUpdate }) => {

 const isCustomSection = sectionId.startsWith('custom-')

 const isDeletable = isCustomSection

 return (

 <div className="section-container">

 <div className="section-header">

 <DragHandle />

 <EditableTitle />

```

```

 <CollapseToggle />

 {isDeletable && <DeleteButton onClick={() => onDelete(sectionId)} />}

 </div>

 <SectionContent />

</div>

)

}

...

```

#### **\*\*Visual Indicators:\*\***

- Custom sections show delete button on hover
- Core sections never show delete option
- Consistent styling regardless of section type

---

## ## 🎮 User Experience Flow

### ### Creation Workflow

#### **\*\*Step 1: Section Addition\*\***

1. User clicks "Thêm mục tùy chỉnh" button
2. New section appears at bottom of editor
3. Default title "Mục tùy chỉnh" is selected for editing

4. Section expands to show empty item form

## **\*\*Step 2: Content Creation\*\***

1. User edits section title to describe content type

2. Clicks "Thêm mục" to add first item

3. Fills in relevant fields (all optional)

4. Adds bullet points as needed

5. Continues adding items or moves to other sections

## **\*\*Step 3: Organization\*\***

1. User can reorder custom sections via drag & drop

2. Can reorder items within sections

3. Can collapse/expand for better navigation

4. Can delete entire sections if no longer needed

## **### Error Prevention**

### **\*\*Safeguards:\*\***

- Confirmation dialogs for all deletion actions

- No accidental loss of content

- Clear visual feedback for all actions

- Undo alternatives through autosave system

---

```
📊 Technical Implementation
```

```
State Management
```

```
Section Creation:
```

```
```typescript
```

```
const addCustomSection = () => {
```

```
  const sectionId = `custom-${Date.now()}`
```

```
  const newSection = {
```

```
    type: 'custom',
```

```
    title: 'Mục tùy chỉnh',
```

```
    items: [],
```

```
    usedAI: false
```

```
  }
```

```
  setCvData(prev => ({
```

```
    ...prev,
```

```
    [sectionId]: newSection
```

```
  )))
```

```
  // Auto-scroll to new section
```

```
  scrollToSection(sectionId)
```

```
}
```

```
...
```



```

**Section Deletion:**

```typescript
const deleteCustomSection = (sectionId: string) => {

 setCvData(prev => {

 const updated = { ...prev }

 delete updated[sectionId]

 return updated

 })

 // Trigger autosave

 saveToDatabase(cvData)

}

```

```

Data Persistence

****Autosave Integration:****

- Custom sections trigger autosave on all changes
- Section addition/deletion saves immediately
- Field updates debounced like core sections
- Consistent with existing autosave behavior

****Export Compatibility:****

- Custom sections included in PDF export
- LaTeX export handles custom sections

```
- Word export maintains formatting

- Section order preserved across formats

---

##  Quality Assurance

### Testing Requirements

**Functional Testing:**

- [ ] Add custom section creates proper data structure

- [ ] Delete custom section removes from CVData

- [ ] Core sections cannot be deleted

- [ ] Custom section items can be added/removed/reordered

- [ ] All field types work correctly (text, dates, bullets)

- [ ] Section reordering works with custom sections

- [ ] Preview renders custom sections correctly

- [ ] Export includes custom sections in all formats

**User Experience Testing:**

- [ ] Intuitive section creation flow

- [ ] Clear distinction between core and custom sections

- [ ] Appropriate placeholder text and examples

- [ ] Responsive design on all devices
```

```
- [ ] Accessibility compliance

- [ ] Error prevention and recovery


**Data Integrity Testing:**

- [ ] No data loss during section operations

- [ ] Proper autosave for all changes

- [ ] Consistent state across page refreshes

- [ ] Export accuracy matches preview exactly


---


## 🚀 Future Enhancements


### Potential Improvements


**Template Library:**

- Pre-built custom section templates for common use cases

- "Certifications", "Projects", "Volunteer Work" starting templates

- One-click addition with appropriate field labels


**Smart Suggestions:**

- Context-aware field suggestions based on section title

- Industry-specific custom section recommendations

- Integration with professional networking data
```

****Advanced Customization:****

- Custom field types beyond the standard work experience model
- User-defined field labels and arrangements
- Section-specific formatting options

Persistent Behavior Across Sections: All sections, whether default or custom, share some common behaviors worth noting:

- Changes in any field auto-save and reflect on preview live.
- Each section (except contact) only shows on the preview if it has content. If the user leaves one entirely empty, it will be omitted from the PDF export. This means users don't have to delete a section if they decide not to use it; they can simply leave it blank and it won't appear, avoiding embarrassing blank sections in the output.
- Section headings on the preview are styled consistently (e.g., all caps or small caps, a certain color or font) following the template design. Renaming a section only changes the text, not the style.
- The user can collapse sections in the editor to focus on others, and those states are independent – collapsing doesn't affect preview content, it's just an editing convenience.
- The interface encourages but does not force the user to complete all sections. If one is incomplete, we may show a nudge in the "Improve Resume" or suggestion indicators, but the user is free to skip it.

CV Score Element – Product Spec

Location

Top-left corner of the Guided Editing interface (sticky, always visible)

Initial Score State

- **Default State:**

- Starts at **0%** if user hasn't entered or improved any CV sections.
 - Clearly labeled as “**Điểm CV: 0%**”.
-

Score Calculation Logic

- Each CV section contributes equally to an **80% maximum score**:
 - **Sections:**
 - Contact Information
 - Professional Summary or Objective
 - Work Experience
 - Skills, Tools, Languages, Technologies
 - Education
 - Volunteer Experience (optional but strongly recommended for freshers)
- Each section comprises smaller components:
 - For example, **Contact Information** includes:
 - Full Name
 - Email
 - Phone Number
 - Location
 - Each mandatory component equally contributes to its section score.
 - Optional fields (e.g., LinkedIn) do not impact score.

Scoring Example:

- **Contact Information (max 13.3% if all sections equal):**
 - 4 components: Full name, email, phone, location.
 - If 1 component is missing: Section score = $(13.3\% \div 4) \times 3 \approx 10\%$.
-

AI-Powered Improvement Requirement

- Manually filling sections has a capped score (**max 80% total across all sections**).
- To exceed this cap (up to 100%), users **must apply AI-generated improvements**:
 - Each section individually unlocks its full potential score **only after using AI suggestions**.
 - Example: Work Experience manually completed = capped at **X%**; applying AI boosts this to full section contribution.
- Alternatively, users can use the premium feature “**Improve All using AI**” to instantly reach maximum potential (100%).

Visual & Gamified Feedback

```
## 🎮 User Experience Patterns

### Gamification Elements

**CV Score System:**
- **Location:** Top-left corner, sticky position
- **Calculation:** 80% from manual entry, 20% from AI improvements
- **Visual Feedback:** Color-coded progress (Red 0-49%, Orange 50-79%, Green 80-100%)
- **Celebration:** Confetti animation at 100% completion
- **Motivation:** Real-time score updates with positive reinforcement

**Progress Indicators:**
- **Section Completion:** Visual progress per section
- **Improvement Opportunities:** Highlight available AI enhancements
- **Achievement Unlocks:** Celebrate milestones and completions
```

- **Real-time Score Updates:**

- Score visually updates immediately upon accepting/rejecting suggestions.
 - Green upward animation (+) for increases.
 - Subtle red downward animation (−) if improvements are undone.
 - **Score Color Coding:**
 - **0–49%:** Red color (indicates significant improvement needed)
 - **50–79%:** Orange/Yellow (moderate, room for improvement)
 - **80–99%:** Green (good, but still can improve using AI)
 - **100%:** Vibrant Green (perfect CV, no improvements needed)
-

Special 100% Score Gamification

- Upon reaching **100%**, a special celebration is triggered:
 - Confetti animation bursts briefly around the score.
 - Score text briefly enlarges, pulses, and glows vibrantly.
 - Friendly congratulatory message displayed:
 - “🎉 Hoàn hảo! CV của bạn đã đạt 100%! 🎉”
- **“Re-evaluate” button:**
 - Available after manual extensive edits for recalculating score accurately.

Drag & Drop Reordering Behaviors

The drag-and-drop functionality in OkBuddy is designed to be intuitive and robust, both for whole-section moves and reordering items within sections. Here we detail how dragging works and how the UI responds:

Section Reordering (Drag & Drop)

- **Initiating a Drag:** The user can initiate a drag of a section by grabbing the 6-dot icon on the section header . The moment the user clicks and holds the handle, the section's card is visually lifted from the layout (z-index raised). We display a *drag ghost* – a semi-transparent version of the section card – that moves with the pointer. The original section's space might collapse or leave an outline placeholder.
- **During Drag:** As the user drags the ghost up or down the list, the editor auto-scrolls if needed (e.g., dragging a section upward at the top of viewport will scroll up to reveal earlier sections). The other sections adjust to indicate potential drop positions: typically we show a horizontal line or gap where the dragged section would slot in. For example, if dragging Summary below Work Experience, a gap opens between Work Experience and Skills indicating “drop here to place Summary”.
- **Dropping:** When the user releases the mouse/button, the section is dropped into the new position. The ghost disappears, the section card snaps into place in the UI, and all section numbering (if any) and ordering is updated. Instantly, the right preview also reorders: the sections in the PDF are rearranged to match. This re-render is triggered immediately on drop (we don't live-update the preview while dragging because the content is still the same, just order changes – it's simpler to update after drop event).
- **Animation:** To make it smooth, we can animate the movement of sections sliding into their new positions and the ghost fading out. The preview panel could also flash the moved section's content or highlight it briefly to confirm the change (especially if the moved section is currently out of view on the preview, we might momentarily scroll or outline it to show where it went).
- **Constraints:** As noted, the Contact Info section is not draggable (in this template). We visually indicate this by maybe not showing a drag handle on that section or disabling it. For other sections, there are no constraints – the user can even put Education above Work Experience if they want. We trust the user's judgment or use-case (e.g., academia might prioritize Education).
- **Keyboard Accessibility:** Although most will use mouse, we also consider keyboard reordering. For the power user or accessibility: when a section header is focused, pressing a certain key combo (like Ctrl+↑ or Ctrl+↓) could move that section up or down in the list. We would announce the new position via ARIA for screen readers. This ensures the feature is available to all users.
- **Error Handling:** It's unlikely to “fail” a drag, but if for some reason a drop doesn't register (glitch), the section will remain in original spot (we ensure no data loss or duplication). Since autosave tracks section order, it'll save the new order immediately on

drop. If a save fails, it might revert on refresh; but we treat that like any save error, prompting a retry.

Entry Reordering within Sections

- **Work Experience Entries:** Each job entry within Work Experience can be repositioned. The UI for this is similar: a small drag handle (maybe 4 dots) on the job entry card's header (usually where the job title is shown). The user drags the job card within the parent section. The behaviors mirror section dragging: other job cards in that section create a placeholder line to show where it will drop. Dropping moves it. The preview's Work Experience section reorders the entries accordingly. If the user had the section collapsed, we might auto-expand it when starting a drag for clarity, or at least show the outline of the dragged item.
- **Education Entries:** Similarly, reorder degrees. Usually sorted by date, but manual override is allowed.
- **Skills:** If we allow manual reordering of skill chips, dragging them will immediately swap position. The preview updates the skill list order. (Though order in skills is not critical, it can reflect priority.)
- **Bullet Points:** For completeness, we allow reordering bullets within a job entry. Each bullet might have a tiny drag handle (like a vertical 3-dot at start) or maybe easier, up/down arrows to shift bullet position. Dragging bullets is a bit more micro, but if implemented, it works the same: drag the bullet text field up or down within its job's bullet list, drop to reposition. The preview bullet order updates. This is helpful if the user wants to prioritize one accomplishment over another easily.
- **Custom list sections:** Any custom section that has multiple items (like Projects) follows the same pattern.

Visual Feedback: We maintain clear visual cues for reordering at all levels:

- The element being dragged always has a ghost image and the original space outlined.
- Drop targets (gaps) are highlighted.
- If dragging near the edges of a container, auto-scroll triggers to scroll that container.
- If dragging an item beyond the top or bottom of its container (e.g., trying to drag a job out of Work Experience), we do not allow cross-section dragging of entries. If user needs to move something to another section, they'd have to create it there or rename sections –

cross-section moves are not logically supported.

- Dragging a section near the preview edge doesn't directly interact with the preview (except maybe highlighting the corresponding preview area), because preview is just output.

Aftermath: After any reorder operation, we autosave the new order in the resume data. The preview (which is essentially a function of the data) is thus always synced with whatever order is now in the left panel. There is no separate “save order” action required.

10× Better – Drag & Drop: Resume.io and similar builders have drag-and-drop, but OkBuddy refines it with smoother feedback and extends it deeply (down to bullet level). The improved aspect is the **fluidity and clarity** of the reordering: animations, live preview highlighting, and the ability to reorder nearly every granular element. This makes organizing the resume a breeze. For example, if the user decides their Skills should be on top (maybe for a skills-based resume), they just drag it to the top and see the preview update – no menu diving or complex toggles. This immediacy and visual confirmation is a significant UX win. Additionally, supporting reordering of bullets and skill tags provides full control to the user to fine-tune content prioritization, which many tools don't allow or make clunky.

Real-Time Preview and Synchronization

A cornerstone of the Guided Editing experience is the always-visible, real-time updated preview of the resume on the right. Here's how it functions and stays in sync:

```
### Rendering Engine Requirements

#### Template Specifications (Dennis Schröder)

**Font & Typography Standards:**

- **Applicant Name:** Calibri, 20pt, Bold, centered at top
- **Section Headings:** Calibri, 12pt, Bold, left-aligned
- **Body Text:** Calibri, 11pt, Regular
- **Secondary Info:** Calibri, 10pt, Italic (dates, locations)
- **Line Spacing:** 1.15
```

- **Margins:** 1 inch (2.54 cm) all around

Professional Formatting Rules:

- **ATS Compatibility:** Standard fonts, clear hierarchy
- **Print Safety:** Margins ensure no content cut-off
- **Screen Readability:** Optimized for digital viewing
- **Consistency:** Uniform styling across all sections

Page Layout & Margin Logic

Professional Spacing Standards:

- **Top Margin:** 0.75 inches from top edge to applicant name
- **Bottom Margin:** 0.75 inches from last line to bottom edge
- **Side Margins:** 1 inch left and right for print safety
- **Section Spacing:** Consistent vertical gaps between sections
- **Breathing Room:** Adequate white space for readability

Page Break Intelligence:

The PDF rendering system implements sophisticated page break logic:

1. **Automatic Page Breaks:**

- Content approaching bottom margin triggers new page
- No text truncation mid-line or mid-paragraph

2. **Keep-Together Rules:**

- Section headings + minimum 2-3 lines of content stay together
- Job entries don't split across pages
- Bullet points remain with their parent job entry

3. **Content-Aware Decisions:**

- **Section Boundaries:** Headings kept with following content
- **Table Handling:** Tables break at row boundaries only
- **List Management:** Bullet lists maintain proper grouping

4. **Margin Enforcement:**

- Defined margins act as "no-content zones"
- Page break triggered before margin encroachment
- Consistent margin preservation across multiple pages

Real-Time Synchronization

Live Update Mechanism:

- **Input Debouncing:** 300ms delay to prevent flicker
- **Incremental Updates:** Only changed sections re-render
- **Focus Synchronization:** Editor selection highlights corresponding preview area
- **Interactive Preview:** Click preview section to focus editor field

Performance Optimization:

```
- **Section-Level Rendering:** Minimize full document regeneration

- **CSS-Based Styling:** Avoid heavy PDF generation during editing

- **Lazy Loading:** Non-visible content rendered on demand

- **Memory Management:** Efficient cleanup of render resources


### Zoom & View Controls


**Zoom Logic:**

- **Default Zoom:** Fit-to-width for constant overview

- **Zoom Options:** 50%, 75%, 100%, 125%, 150%

- **Zoom Persistence:** Maintain zoom level during session

- **Auto-Fit:** Automatically adjust for optimal viewing


**Navigation Controls:**

- **Page Indicators:** Show current page / total pages

- **Scroll Sync:** Smooth scrolling within preview area

- **Section Jump:** Quick navigation to specific sections

- **Full-Screen Mode:** Expand preview for detailed review

- Location: Subtle top right corner of the CV being displayed


### Content Visibility Rules


**Dynamic Section Display:**

- **Empty Section Hiding:** Sections with no content don't appear in preview
```

```
- Heading Suppression: No empty headings rendered

- Smart Formatting: Contact info renders without "Contact" heading

- Consistent Styling: All sections follow template formatting rules


Content Validation:

- Required Field Checking: Name field must be present

- Format Validation: Email, phone number format verification

- Date Logic: Chronological consistency enforcement

- Length Optimization: Content length warnings for readability
```

Preview Rendering Engine

The preview panel renders the resume using the **Dennis Schröder template** design. This template's key style points (for clarity):

- Contact info is top-centered: the Name appears in a large font, centered, and the contact details in smaller font below it, also centered.
- Section headings like "Summary", "Work Experience", etc., are likely distinctive (perhaps bold or colored) and left-aligned (unless the template centers them – but likely left).
- Work Experience entries: as specified, title/company/location on one line, dates on second line, bullets indented under each.
- Skills: possibly a multi-column or running list of keywords.
- Education: similar style to work entries without bullets.

We ensure the preview uses the exact font families, sizes, and spacing defined by the template. It is essentially a mini PDF viewer or an HTML representation that mimics the PDF.

Implementation detail: The preview could be an <iframe> canvas that updates content, or an in-browser PDF renderer that regenerates on the fly. Performance-wise, re-generating a PDF with each keystroke might be heavy, so likely we use an HTML/CSS that is template-styled (this can then be exported to PDF when needed). The user shouldn't tell the difference – it looks like the final output.

Live Update Mechanism

- **Two-way link:** The preview listens for changes in the resume data model. Our editor is likely built on a framework that binds form inputs to a data object. On each update (e.g., user types a character in the Summary), the data model updates and triggers a re-render of the preview component for that text. This happens in near real-time (with a small debounce to avoid flicker on every single keystroke – maybe update after the user pauses for 300ms, or on field blur for less critical fields).
- **Text Updates:** As soon as a field is updated, the corresponding text in the preview changes. For example, typing “Man” in a bullet will show “Man” in that bullet on the preview, letter by letter. If this causes line breaks or page breaks to shift, the preview adjusts layout accordingly live.
- **Section Addition/Removal:** If a section is deleted in the editor, the preview instantly removes that entire section (the content and heading). If a section is added or reordering, the preview adds or moves it immediately. There might be a slight fade in/out effect for add/remove to make it noticeable.
- **Formatting and Layout:** The preview ensures things like long text wrapping, multiple pages, etc., are handled. If the resume content goes beyond one page, the preview will show a second page (vertical scroll within the preview area, or maybe a pagination control). The user can see exactly where the page break happens. We may optionally show a dashed line or some indicator for the page break in the preview, which some builders do in edit mode to warn if you spill onto page 2. This helps users aiming for one-page resumes to know they exceeded one page.
- **Synchronization of Focus (Optional):** When the user selects a section on the left, we could highlight the corresponding section on the right (e.g., a light yellow highlight behind that section text) to visually connect where they are editing. Similarly, as mentioned, if the user clicks on text in the preview, we find which section/field that corresponds to and focus that field in the editor (and maybe expand that section if it was collapsed). This synergy speeds up navigation and makes the preview interactive. It’s a significant UX improvement to allow clicking the preview to edit, which feels natural (many would attempt to click directly on the preview text expecting to edit; we catch that event and redirect it to the form).
- **No separate Preview step:** Unlike some tools that have an “edit mode” and “preview mode”, OkBuddy is always in preview mode essentially. Users don’t need to toggle or open a preview manually. This saves time and encourages constant alignment with final output.

Performance Considerations

Because of constant updates, we ensure the preview rendering is optimized:

- Minor text changes reflow only that section's text, not the whole document from scratch if possible.
- If using HTML, use CSS classes for formatting rather than recalculating styles each time.
- If using PDF generation behind scenes, perhaps generate once per second of idle rather than on every char.
- In most cases, text changes are small so it's fine. Dragging sections doesn't actually change content, just order, so that's a quick reflow.
- When applying a big AI suggestion (like rewriting all bullets), that might cause a lot of text to change at once. The preview will update correspondingly. We handle it gracefully (no jarring flicker, ideally content just morphs into the new text).
- The preview panel is scrollable independently. The user might scroll down on the preview to see the second page while editing something on first page. We do not auto-scroll preview on every input (that could annoy if they are proofreading page 2 and editing page 1). We only scroll preview under specific interactions: e.g., if user clicks a preview item or if they add content that creates a new page, maybe jump to show it.
- The preview uses high-resolution as much as possible so that what you see is print-quality (especially for PDF). But during editing, we might use a slightly lower resolution (for performance) and render full quality on demand or at final export. The trade-off: we want crisp text and layout but if images or icons are part of template, maybe show a placeholder or low-res and sharpen later. Given Dennis template likely minimal graphics, not a big issue.

Template-specific Behavior

The Dennis Schröder template has a certain visual hierarchy:

- The Contact Info is centered – our preview implementation ensures that if the user's name is long or short, it still centers properly. Also if one of the contact fields is missing, we adjust punctuation (for example, if location is missing, we don't print the trailing “•” separator after phone).
- Work Experience and Education headings might use small caps or color. We replicate that exactly for any renamed title too (if user renames Work Experience to “Experience”,

it still gets the styling).

- Font consistency: All text comes from the template's fonts. We embed them or reference them so the preview matches exactly the PDF output.
- Margins and spacing: The preview is a true representation – e.g., if the template has a top margin of X for contact section, we show that whitespace. Essentially the preview looks like a scaled version of the printed resume.

Interactivity and User Feedback

- The user can zoom or enlarge the preview if needed. Perhaps a zoom control (e.g., 100%, 150%) or a button to open the preview in full-window or PDF viewer separately. This doesn't affect editing, just for a closer look.
- The preview is not directly editable (clicks are redirected as said). We also ensure that if the user double-clicks text in preview out of habit, we perhaps highlight the corresponding field or simply do nothing besides indicating that's not the edit area.
- We might overlay the preview with a transparent layer to catch click events and translate them to edit actions, to prevent any default selection behavior, etc.
- If the preview fails to load (e.g., an image asset missing or a font not loaded), we show a fallback message or try to recover. Most likely, if offline, the preview might not update if it relies on an API; but we should make it client-side to avoid that.

10× Better – Preview: Many resume tools have a live preview, but OkBuddy's is **truly real-time and interactive**. The user doesn't need to guess how inputs map to output – it's continuously visible. The ability to click the preview to edit and instant feedback loops make the process feel seamless. This eliminates the tedious switch-and-check dance users often had to do (for instance, older systems might require hitting a Preview button and then going back to edit if something looked off). OkBuddy's preview also smartly syncs content inclusion (hiding empty sections) and indicates page length, guiding the user to produce a well-formatted resume without nasty surprises. This real-time WYSIWYG approach is a massive improvement over any static or delayed preview.

Input Validation & Error Handling

To ensure the resume is not only well-written but also technically sound and professional, the editor includes various validation rules and error-handling mechanisms. These operate in real-time as the user inputs data, providing immediate feedback and guidance.

****Critical Anti-Pattern Prevention:****

```
- **NEVER use inline validation functions** (causes infinite re-renders)

- **ALWAYS use useCallback** for validation functions

- **Implement safety timeouts** to prevent infinite loops

- **Debounce validation** with 300ms delay
```

Field-specific Validation

- **Name (Contact Info):** Cannot be empty (required). If left blank, the field border turns red and an error message “Name is required” appears. We also disallow certain characters (e.g., numeric digits, as they are likely accidental in a name). If the user enters something like “john doe” in lowercase, we might gently suggest capitalizing it (“John Doe”) – possibly auto-capitalize on blur, or show a suggestion to do so (AI or simple rule).
- **Email:** Must match a basic email regex pattern. On blur or while typing, if the pattern is invalid, we underline it red. If it lacks “@” or a domain, we show “Invalid email address.” We also handle common typos: if the user types “@gmial.com”, we could suggest “Did you mean @gmail.com?” in a tooltip (the user can click to accept the correction). This uses a small dictionary of common typos – a helpful touch.
- **Phone:** We strip out spaces and dashes internally to validate numeric content. If after removing those, there aren’t enough digits (say <7 digits) or it contains letters, we flag it. We might also auto-format a bit: e.g., for US numbers or known country formats, or at least ensure consistent spacing. If a phone number is extremely long (>15 digits), likely error unless it’s an international number with country code (we allow + and up to maybe 15 digits as per international standards). If invalid, error “Please enter a valid phone number or include country code.”
- **Location:** No strict validation, but we keep an eye on length (shouldn’t be a full address). If the user enters multiple commas (like full address), we might suggest just city/state or city/country to keep it concise.
- **Dates (Work Exp/Education):** We ensure correct chronology. If an End Date is provided and is earlier than the Start Date, we immediately show an error on that field: “End date is earlier than start date.” And possibly don’t allow that state (maybe auto-swap them or prevent selecting a wrong year in dropdown).
 - If present checkbox is selected, we ensure End Date field is cleared or disabled.

- If a date is outlandish (like year 1900 for a modern resume, or 2100 in the future), a subtle warning could appear (“Check the year – is this a typo?”).
- The format is standardized (Month YYYY). If user tries to manually type and it doesn’t match, we guide them to use the picker or format correctly.
- **Bullet Points & Text Areas:** We generally allow free text but we check:
 - Spelling mistakes: the editor underlines spelling errors (utilizing the browser’s spellcheck or an integrated dictionary).
 - Basic grammar: AI could underline passive voice or very long run-on sentences, offering improvements (like a Grammarly-lite feature embedded).
 - If a bullet or summary sentence ends without punctuation when it should, we might suggest adding a period (though in resumes, bullet points often don’t have periods – we follow conventional style, maybe no period needed for bullets).
 - Prohibited content: We might ensure no profanity or sensitive info is inadvertently included, though that’s more user’s responsibility. Possibly a check if the user copy-pasted an entire job description into their resume (some might do that mistakenly) – we could warn “Your text is very similar to a job posting – resumes should describe your experience, not the job listing.”
- **Character Limits:** While we don’t hard limit content, extremely long entries can break formatting. If a user writes an essay in Summary (say 500+ words), we could either truncate on preview or advise them to shorten. We provide visual cues as described with word count. Similarly for any field that is too lengthy (like a skill chip text > 3 words likely should be something else).
- **Unique Section Rules:**
 - Skills: no duplicates allowed (we quietly prevent adding a skill if it exactly matches an existing one). Also, if skill name is very uncommon or possibly a typo (AI can check if it doesn’t match known skill terms), we could underline it. E.g., user types “Javascript” – not recognized, suggest “JavaScript”.
 - Custom sections: validation would depend on the field type (similar to above rules).
 - If user inputs an obvious placeholder or filler text like “Lorem ipsum” or “asdf”, we might alert “It looks like placeholder text – don’t forget to replace it with actual info.”

Error Indicators and Messaging

Our approach is to **catch issues early and show non-intrusive inline messages**:

- Each field can show an error message directly beneath it in a small red italic font if invalid. The field itself is outlined or underlined in red.
- The section header might also carry an alert icon if something inside is invalid or missing crucial info. For example, if the Summary is empty, maybe not an “error” per se, but if we consider it recommended, the Improve panel could mark it. But required fields like Name definitely cause an alert on Contact section header until resolved.
- We avoid modals or blocking popups for validation (except when absolutely needed, like confirming deletion or leaving page with unsaved changes, though autosave mitigates the latter).
- Error messages are phrased constructively, e.g., “Please enter your name” rather than just “Required”.

If multiple errors exist, a user can still continue editing; we don’t stop them. But we might implement a **final check** if they try to export/download: if Name is empty or email invalid, at that point we might pop up “Before finishing, please fix these issues: [list]”. But since this is about editing UI, within the editor we simply highlight issues.

```
#### Error State Management
```

```
**Field-Level Error Handling:**
```

- **Visual Indicators:** Red border, error message below field
- **Section-Level Alerts:** Alert icon on section header
- **Non-Blocking Approach:** Users can continue editing with errors present
- **Constructive Messaging:** "Please enter your name" vs "Required"

```
**Date Validation Logic:**
```

- **Chronological Consistency:** End date must be after start date
- **Current Employment:** "Present" checkbox disables end date

```
- **Future Date Prevention:** No dates beyond current month (except "Present")  
- **Format Standardization:** Month YYYY format enforced
```

AI-Assisted Error Correction

Where possible, we use AI not just to point out problems but to suggest fixes:

- Misspelled words: right-click (or a suggestion UI) could offer the correct spelling.
- Poor grammar: an “Improve” suggestion can fix it as part of style improvements.
- If a user writes something very long-winded, AI might suggest a concise alternative automatically.
- If a field that should be capitalized (names, proper nouns, etc.) is lowercase, AI or rules can auto-correct it.
- If information is inconsistent (e.g., in one job bullet they say “Managed 10 people” and in another job they say “Managed a team of 8” but later mention leading 10 again – AI might not catch that context, but just an example of where advanced consistency checks could go. That might be out-of-scope for now).

Handling AI Errors

We also consider error handling in AI operations:

- If an AI suggestion fails to load or the API call errors out (network issue, etc.), we catch it and show a friendly message in context: e.g., when clicking “Generate Summary,” if it fails, show “Sorry, the AI is currently unavailable. Please try again.” perhaps with a retry button.
- If the AI returns content that doesn’t seem right (maybe it misunderstood input or produced something irrelevant), the user should be encouraged to try again or refine input. We can have a thumbs-down feedback on suggestions which could in future improve the system, but in immediate terms, we could then offer a different approach (like if “Generate bullets” gives a weird result, maybe prompt the user “That didn’t seem helpful – try providing a bit more detail about your role for better suggestions.”).
- We also ensure that applying suggestions doesn’t override user content without consent. For instance, “Apply All” should have either an undo or a confirmation if it’s about to change a lot. Possibly a toast “All suggestions applied – Undo” that lasts for say 5-10 seconds.

- If an AI rewrite makes a factual assumption (common with generative AI, e.g., adding a metric that the user didn't provide), we highlight that change. For example, if the AI said "increased sales by 50%" and the user never gave that number, we highlight it and ask for confirmation. The user should verify such facts. This is tricky to automate but at least in user training we caution them to verify all content.

Save/Load Errors

- If autosave fails (due to connectivity), the UI should clearly indicate "Saving failed - retrying..." and perhaps an option to manually retry or download a backup. The data should remain in local memory so the user doesn't lose it. Once connection is back, it will save. If the user tries to navigate away during a save failure, a warning "Changes are not saved. Are you sure you want to exit?" will appear.
- Version conflicts (probably not an issue unless real-time collaboration is a thing, which it isn't here).
- If the preview fails to render for some reason, we might show a fallback textual preview (just listing sections in plain text) or an error message on preview "Preview unavailable. Please refresh." But given it's a core feature, we'd try to prevent that.

Error/Validation Summary (if needed): Perhaps in the "Improve resume" or a top status bar, we might list how many issues remain (like "3 issues to fix"). The user could click that to jump through them. This is similar to a lint check or completeness score (Resume.io had a completeness score concept). OkBuddy could incorporate a "Resume health" meter that checks for missing sections, too-long content, etc. While not explicitly requested, it could be part of the recommendations panel. For now, we handle issues inline primarily.

10x Better – Validation: OkBuddy's error handling goes beyond red underlines – it actually helps fix the errors. By integrating suggestions for corrections and ensuring no critical detail is overlooked (ex: missing name or mismatched dates), it provides a safety net that standard editors lack. Resume.io would notify about missing keywords or length maybe, but OkBuddy proactively fixes formatting and common mistakes (like capitalization or email typos) with minimal user effort. This intelligent validation means users end up with a cleaner, more professional resume by default. It's like having an editor proofread your resume in real-time – a big improvement in user experience and outcome quality.

Export

```
##  Export Features

### Multi-Format Export Support
```

```

**High-Quality PDF Export:**
- **Resolution:** 300 DPI for print quality
- **Color Space:** RGB for digital, CMYK option for print
- **Font Embedding:** Ensure consistent rendering across devices
- **Compression:** Optimized file size without quality loss
- **Metadata:** Include title, author, creation date

**Microsoft Word Export:**
- **Format:** .docx with full formatting preservation
- **Editability:** Maintain text editability post-export
- **Template Compatibility:** Consistent with Word's resume templates
- **Font Fallbacks:** Handle missing fonts gracefully

**LaTeX Export:**
- **Template:** Professional moderncv package
- **Language Support:** Vietnamese language compatibility
- **Character Escaping:** Proper handling of special characters
- **Customization:** User-modifiable LaTeX source
- **Professional Output:** Publication-ready formatting

### Export Quality Standards

**Visual Fidelity:**
- **WYSIWYG Guarantee:** Export matches preview exactly
- **Cross-Platform Consistency:** Identical rendering across devices
- **Font Accuracy:** Consistent typography in all formats
- **Layout Preservation:** Maintain spacing and alignment

**File Optimization:**
- **Size Efficiency:** Optimal file sizes for sharing
- **Loading Speed:** Fast generation and download
- **Compatibility:** Works across all major platforms
- **Accessibility:** Screen reader compatible formats

```

AI Logic & Section-Specific Smart Features

One of the defining aspects of this Guided Editing interface is the deep integration of AI at every level.

```

**Smart Internal Library Priority:**

```

```
1. **Check Internal Library:** Match job title against curated bullet library
2. **Library Hit:** Display pre-vetted, high-quality suggestions
3. **Library Miss:** Fall back to AI generation with structured prompts
4. **Cost Optimization:** Reduce API calls by 70-80% through library usage
```

Here we specify the AI-driven features by category and how they elevate the user experience:

AI Keyword Targeting (Resume Optimizer for Job Description)

```
**Keyword Analysis Pipeline:**
1. **Parse Job Description:** Extract skills, responsibilities, technologies
2. **Map to Sections:** Skills → Skills section, tools → Experience bullets, traits → Summary
3. **Gap Analysis:** Compare current CV content against required keywords
4. **Contextual Suggestions:** Generate section-specific integration suggestions
5. **One-Click Application:** "Apply All" applies all suggestions intelligently
```

As discussed, users can paste a job description into the system to tailor their resume to that job. Under the hood:

- The AI parses the job description for **keywords** (skills, job-specific verbs, qualifications, technologies, etc.) and possibly **keyword frequency** to gauge importance.
- These keywords are mapped to sections of the resume:
 - If a keyword is a skill (like “Excel”, “JavaScript”), it maps to the Skills section.
 - If it’s a responsibility or tool used in work (like “budget management” or “Salesforce”), it maps to Work Experience (the AI will look to incorporate it into one of the job bullets, ideally where context fits).
 - If it’s a soft skill or overarching trait (like “detail-oriented”), it might map to Summary or Cover Letter (if we had cover letter). For resume, summary is a place for such traits.
 - If it’s an educational requirement (like “MBA”), maps to Education section.
- The AI also assesses the resume’s current content to see which of those keywords are already present and which are missing. Perhaps it even computes a match percentage similar to Jobscan’s resume score , but primarily we use it for suggestions rather than a

raw score.

- For each missing keyword or concept, an **AI suggestion is generated** on how to include it. This is important – we don't just list the word, we provide a context or sentence. For example, missing keyword "project management":
 - In Summary: "... skilled in project management and cross-functional team leadership."
 - In Work Experience (if a job bullet can fit it): "Led multiple projects simultaneously, demonstrating strong project management skills."
 - In Skills: simply suggest adding the skill tag "Project Management".
- We display these suggestions at their points of use, as described (via icons or inline highlights). The user can accept, modify, or ignore each.
- If the user hits "Apply All" for the job tailoring suggestions, the AI will intelligently merge all of them at once:
 - It might combine multiple keywords into one sentence if possible to avoid bloat. Or distribute them across different sections if logical.
 - It ensures the text still reads naturally (this is a challenge if adding many keywords blindly; the AI's language skills handle it).
- After applying, we might recalc the match; ideally the resume should now contain most of the important terms from the job posting. This feature directly addresses the ATS filtering issue by aligning with how 75% resumes are filtered .
- This logic essentially replicates and enhances the Resume.io "keyword suggestions from job posting" update , but in OkBuddy it's integrated per section and can auto-apply changes. It's like having a built-in Jobscan or ATS optimizer within the editor.

Content Generation & Suggestions (Per Section AI)

```
**Suggestion Integration Logic:**  
- **Skills Section:** Direct tag addition  
- **Experience Section:** Contextual bullet modification  
- **Summary Section:** Natural phrase integration  
- **Education Section:** Certification/course suggestions
```

We've outlined these in each section, but to recap the major tools:

- **Summary Writer:** A GPT-based function that can generate a summary from scratch. It takes into account user inputs (like titles, years experience). Possibly the user can provide a one-line prompt “I want to emphasize my sales and technical skills” to guide it. It outputs a paragraph. The logic filters out any first-person pronouns if not desired (some prefer no “I” in resume, etc.).
 - Prompt: *"Using the following job description '{JD}', user's professional background '{work_experience}', and key skills '{skills}', generate a concise, engaging professional summary in 50-80 words, optimized for ATS compatibility and recruiter appeal."*
 - Output: Single, well-formulated paragraph.
- **Bullet Generator for Work Experience:** The system prioritizes using the curated Smart Internal Library - Job Description Bullet Suggestions for generating resume bullet points to ensure high-quality, vetted suggestions and significantly reduce API costs. If the user's provided job title matches one within this pre-defined library, relevant bullet points are immediately displayed from this library without invoking the AI. Only when the job title is not available in this predefined set does the system then prompt the AI with the following structured template:
 - Prompt (when fallback is necessary): *"Given job title '{job_title}', company '{company}', and responsibilities '{brief_input}', create 3-4 impactful, results-oriented resume bullet points that include quantifiable achievements or relevant keywords from the job description '{JD}'."*
 - Achievements Focus: The AI metric booster logic possibly uses a secondary model or rules: For each bullet, check for numbers or results. If none, attempt to generate a follow-up clause that could be added. Since AI can't guess actual numbers, it might propose placeholders or generic outcomes (“... resulting in improved efficiency.”). We prefer to prompt the user for data to fill in, as described, to keep truth.
 - Output: A concise, actionable list of bullet points highlighting measurable achievements and skills relevant to the targeted job.
- **Skills Section Suggestions**
 - Prompt: *"From the provided job description '{JD}' and existing user profile '{work_experience, education}', identify relevant professional skills or technologies missing from '{current_skills_list}' and suggest up to 5 additional skills relevant to the job."*
 - Output: List of suggested skills.
- **Bullet Point Improvement (Metric Booster)**
 - Prompt: *"Improve the following resume bullet point '{existing_bullet}' by adding a specific measurable result or quantified outcome relevant to '{job_title}', without fabricating numbers or facts."*
 - Output: Enhanced bullet point, marked clearly for user review.
- **Content Rephrasing & Optimization:** The system uses this prompt when users click & use the feature “Improve” (“Cải thiện”) on any part of the resume.

- **Prompt:** *"Rephrase the following resume content '{content}' to improve clarity, conciseness, and ATS optimization while preserving the original meaning and factual accuracy."*
- **Output:** Rewritten text, optimized for readability and ATS.

Autosave & State Logic

```
**Continuous Save Logic:**
- **Trigger Events:** Field blur, significant changes, AI applications
- **Debounce Timing:** 2-second delay for typing, immediate for major changes
- **Error Handling:** Retry logic for failed saves, local backup
- **Status Indication:** Subtle "Saving..." and "Saved" indicators

**State Persistence:**
- **Session Storage:** Maintain state during browser session
- **Auto-Recovery:** Restore content after unexpected closure
- **Version History:** Track major changes for potential undo
- **Cross-Device Sync:** Maintain consistency across devices
```

- The autosave uses likely an API endpoint to save the resume JSON. It triggers after any significant change with a slight delay. If the user is typing continuously, we might wait until they stop for a second, then save.
- The system could maintain a local backup (localStorage) so that if the page refreshes unexpectedly, the last state is not lost even if the network hadn't saved it yet.
- Each autosave on server might create a version (so we could implement undo beyond just immediate actions, but that might be future).
- The user doesn't see this beyond the small indicator. The design ensures saving is quick and unobtrusive (we don't freeze the UI during save).
- When AI functions modify content, those also trigger autosave after applying results.

Default vs. Empty Section Logic

- This we covered: sections hidden on preview if empty.
- On the editor side, empty sections show helpful placeholders or are collapsed with a note "(Empty)".

- If a user tries to download a resume with an obviously empty major section (like no Work Experience at all and they didn't delete the section), we might warn or at least ensure the section heading isn't shown on PDF. We will have the code not render headings if no content items to avoid an empty heading hanging there.
- The default provided sections mean even if empty, the user sees them in editor and can choose to delete if not needed.

Section-Specific AI Recap

- **Summary:** Generate, improve current.
- **Work Experience:** Generate bullets, improve bullets, add metrics, keyword injection.
- **Skills:** Suggest skills, check relevance.
- **Education:** Autocomplete proper names, suggest adding honors.
- **Custom Sections:** Use context to either treat them like Work Exp (if list) or Summary (if text) or Skills (if tags). For instance, if user adds "Certifications" list, we might not have AI content to generate since it's usually just a name of cert and date. But we could suggest popular ones if user's field is known (like AWS Certification for devs).
- **Cover Letter (if outside scope):** It was mentioned resume.io has cover letter generator. Possibly out of this spec's scope, but if integrated, likely another section or separate flow.

"10× Better" Summary for AI:

OkBuddy's AI-first approach imbues every editing action with intelligence. Instead of AI being a separate optional tool (as was the case in Resume.io's "Improve Resume" panel) , here it's woven into the user's workflow. This constant assistive presence is transformative: the user gets the sense of a smart assistant guiding them, rather than a static form. The improvements are tangible – a process that used to take hours (thinking of what to write, how to phrase, how to tailor to a job) can now be accomplished in minutes, with higher quality output. The AI not only speeds things up, it also educates the user (for instance, by example – the user learns what a good bullet point looks like, or which skills matter).

Specifically, features like **Smart Compose for resumes** (predicting text as you type) are cutting-edge; they leverage the familiarity of email predictive text but for professional writing. The one-click optimizations (Apply All, rewrite entire sections) save repetitive labor and ensure consistency in tone and style across the document, which is hard to maintain when writing manually over days or weeks.

In short, the integrated AI is like having an expert co-author, an ATS coach and a reliable buddy built into the editor, making OkBuddy's Guided Editing experience far superior to a traditional resume builder. It addresses users' pain points (hate of typing, uncertainty about wording, fear of missing ATS keywords) head-on with concrete solutions.

AI API Integration Guideline

- **API Provider:** Custom ChatGPT-based API provided upon request. (Cursor or engineer needs to prompt me, OkBuddy's owner for this)
- **Communication:** REST API using secure HTTPS requests/responses with API keys authentication.
- **Format:** JSON input/output for structured data exchange.
- **Throttle:** Rate-limiting to manage cost and prevent overuse (user-visible feedback if rate-limiting occurs).
- **Error Handling:** Graceful user notification if API responses fail or return errors (e.g., retry options or gentle fallbacks).

Impact & Cost Optimization Considerations

Token-Efficient Prompt Engineering:

- All prompts are structured concisely to minimize unnecessary tokens.
- Information is passed efficiently: context provided only as required.

Intelligent Caching:

- Frequently used or repeated AI outputs (e.g., skills suggestions for common roles, industry-specific keywords) cached internally.
- Cached content reduces API hits, saving cost significantly.

Strategic AI Triggering:

- AI improvements are explicitly triggered by user actions (e.g., button clicks: "Generate bullets," "Improve summary").

- Reduces unnecessary API calls by avoiding automatic triggering unless clearly required.

User-Directed Prompt Refinement:

- Allow users to specify minimal context to refine AI-generated content without generating entire new outputs unnecessarily (e.g., prompt refinement: “Make bullet more concise,” “Emphasize technical skills”).

Graceful Handling of API Failures or Limits:

- User notified clearly but gently if API rate-limit reached or if there’s a service interruption.
- Provide clear retry options or local caching suggestions to maintain user experience without disruption.

AI Quality Assurance

```
**Content Quality Standards:**

- **Relevance Accuracy:** AI suggestions match user context

- **Professional Tone:** Appropriate language for business context

- **Factual Safety:** No fabricated metrics or false claims

- **Grammar Standards:** Error-free generated content


**Integration Testing:**

- **API Reliability:** Consistent AI service availability

- **Fallback Mechanisms:** Graceful handling of AI failures

- **Rate Limit Handling:** Smooth user experience within limits

- **Content Caching:** Efficient reuse of previous AI results
```

Smart Internal Library - Job Description Bullet Suggestions

Here are the **15 most common and representative job titles** across various fields relevant to OkBuddy's global user base, along with **3 diverse, impactful bullet point suggestions** for each role. These suggestions are carefully tailored to provide clear, results-oriented descriptions suitable for immediate insertion into users' resumes.

1. Marketing Manager

- "Developed and executed marketing campaigns that increased lead generation by 40%."
- "Managed a team of 5 marketing specialists, providing guidance and performance feedback."
- "Analyzed market trends and customer data to inform product positioning and strategy."

2. Software Engineer

- "Designed, developed, and deployed scalable web applications, reducing load times by 25%."
- "Collaborated with cross-functional teams to define, build, and implement technical solutions."
- "Optimized codebases for performance improvements, reducing infrastructure costs by 15%."

3. Sales

- "Exceeded quarterly sales targets by 30%, generating significant revenue growth."
- "Built and maintained relationships with key clients, enhancing repeat business rates by 20%."
- "Delivered persuasive presentations, effectively converting leads into customers."

4. Business Development

- "Identified and secured new business opportunities, increasing company revenue by 25%."
- "Negotiated partnerships and strategic alliances, enhancing market presence and growth."
- "Implemented customer retention strategies, reducing client churn by 15%."

5. Project Manager

- “Led cross-functional teams to deliver projects on time and within budget, reducing costs by 10%.”
- “Implemented Agile methodologies, improving project completion rates by 20%.”
- “Managed project scope, risks, and communication, resulting in high stakeholder satisfaction.”

6. Data Analyst

- “Analyzed large datasets to identify trends and insights, informing strategic business decisions.”
- “Created interactive dashboards and reports, improving data visibility and decision-making speed.”
- “Streamlined data collection processes, reducing report preparation time by 30%.”

7. Customer Success Manager

- “Developed onboarding and support programs, increasing customer retention by 25%.”
- “Conducted regular check-ins and collected feedback, significantly enhancing client satisfaction.”
- “Facilitated cross-selling and upselling strategies, boosting account revenue by 15%.”

8. Human Resources Specialist

- “Managed recruitment processes, reducing time-to-hire by 20%.”
- “Developed and implemented employee engagement initiatives, increasing retention rates.”
- “Ensured compliance with labor laws, mitigating legal risks and workplace disputes.”

9. Graphic Designer

- “Created compelling visual designs that boosted client engagement by 30%.”
- “Managed multiple design projects simultaneously, consistently meeting tight deadlines.”
- “Collaborated with marketing teams to develop visuals aligned with brand strategy.”

10. Product Manager

- “Led product roadmap development and feature prioritization, increasing user satisfaction by 25%.”
- “Conducted market research and competitor analysis, informing strategic product improvements.”
- “Coordinated cross-functional teams to deliver product launches ahead of schedule, increased revenue by 30%.”

11. Operations Manager

- “Streamlined operational processes, reducing overhead expenses by 20%.”
- “Developed and implemented quality control procedures, enhancing service delivery efficiency.”
- “Oversaw supply chain logistics, ensuring timely delivery and inventory accuracy.”

12. UX/UI Designer

- “Designed intuitive user interfaces, enhancing user engagement metrics by 35%.”
- “Conducted user research and usability testing, leading to significant product improvements.”
- “Collaborated with developers to ensure seamless implementation of design specifications.”

13. Content Marketing Specialist

- “Developed and executed content strategies that boosted organic traffic by 40%.”
- “Created SEO-optimized blog posts and articles, improving search rankings and visibility.”
- “Managed editorial calendars, ensuring timely and consistent content publication.”

14. Financial Analyst

- “Performed detailed financial analyses and forecasts, guiding investment decisions.”
- “Identified cost reduction opportunities, resulting in savings of up to 15% annually.”
- “Prepared comprehensive financial reports, enabling informed executive decisions.”

15. Account Manager

- “Maintained and expanded client relationships, increasing renewal rates by 20%.”
- “Served as primary client liaison, effectively resolving concerns and enhancing satisfaction.”
- “Analyzed client accounts to identify growth opportunities, leading to increased revenue.”

Why These Titles and Bullets?

- These **15 roles** represent the vast majority of OkBuddy’s anticipated user base, covering fields such as technology, sales, marketing, operations, and customer management.
- The provided bullet points emphasize measurable outcomes, action verbs, and relevance to recruiters—directly addressing common pain points for resume writers, such as difficulty quantifying achievements or clearly articulating responsibilities.

- The diversity of each set ensures users can find at least one or two highly relevant bullets, drastically reducing the need for extensive manual typing and editing.

This structured library will significantly streamline the user experience, ensuring high-quality, impactful resume descriptions with minimal effort from the user.

Responsive Design Patterns

```
### Responsive Design Patterns

**Desktop Experience (Primary):**

- **Two-Panel Layout:** Editor left, preview right
- **Drag & Drop:** Full reordering capabilities
- **Keyboard Shortcuts:** Power user navigation
- **Multi-Monitor:** Optimized for wide screens

**Tablet Adaptation:**

- **Stacked Layout:** Preview below editor
- **Touch Optimized:** Larger touch targets
- **Swipe Navigation:** Gesture-based section switching
- **Simplified UI:** Essential controls only

**Mobile Considerations:**

- **Single Column:** Full-width editing experience
- **Modal Preview:** Preview opens in overlay
- **Simplified Input:** Streamlined form fields
- **Touch-First:** Mobile-optimized interactions
```

Other Technical Implementation Requirements

```
## CV Guided Editing Components

### Core Components
- CVEditor: Main orchestrator component managing CV data, score calculation, and autosave
- Header: Top navigation with OkBuddy logo, CV score display, autosave status, and user avatar
- EditorPanel: Left panel containing all CV sections with drag-and-drop functionality
- PreviewPanel: Right panel with live WYSIWYG preview of CV
- ScoreIndicator: Progress bar component showing CV completion percentage with color coding

### Section Components
- ContactSection: Contact information form with validation
- SummarySection: Professional summary text editor
- WorkExperienceSection: Multiple job entries with bullet points
- SkillsSection: Tag-based skill management interface
- EducationSection: Academic background entries

### Utility Components
- DraggableSection: Wrapper component enabling section reordering via @dnd-kit
- AIAssistButton: Action button for AI-powered content generation
- AutoSaveIndicator: Dynamic save status display with Vietnamese text and visual feedback

### Autosave System Components
- useAutosave Hook (`/utils/useAutosave.ts`): Comprehensive autosave management
  - Debounce Saving: 2-second debounce with reset on rapid changes
  - Status Management: 'idle', 'saving', 'saved', 'error' states with transitions
  - Error Handling: Retry logic with configurable attempts and delays
  - Auto-Reset: Automatic return to idle state after save completion
  - Force Save: Manual save trigger for immediate persistence
  - Memory Management: Proper cleanup of timers and resources
- Supabase Client (`/utils/supabase.ts`): Database persistence layer
  - Lazy Initialization: Build-time compatible client setup
  - Mock Fallback: Development mode with simulated saves
  - CV Operations: saveCVData and loadCVData functions
  - Error Handling: Comprehensive error responses with user messages
- AutoSaveIndicator Component: Dynamic UI for save status feedback
```

- **Status Display**: Orange saving, green saved, red error, gray idle
- **Vietnamese Text**: "Đang lưu...", "Đã lưu tự động", "Lỗi lưu tự động"
- **Icons**: Spinning loader, checkmarks, error icons
- **Time Formatting**: "vừa xong", "X phút trước", "X giờ trước"
- **Error Recovery**: Retry button with tooltip error details

Data Flow Architecture

- **CV Data Management**: Centralized state in CVEditor using React hooks
- **Score Calculation**: Real-time calculation via calculateCvScore utility on data changes
- **Live Preview**: Immediate reflection of changes from EditorPanel to PreviewPanel
- **Section Updates**: Individual section updates propagate through onUpdateSection callback
- **Drag & Drop**: Section reordering via @dnd-kit with immediate state updates
- **AI Integration**: Prepared for AI features with usedAI flags per section
- **Autosave Flow**: CV data changes → useAutosave debounce → Supabase persistence → status feedback
- **Error Recovery**: Save failures → retry logic → error status → manual retry option

Score Calculation System

- **Algorithm**: Equal section weights (20% each) for Contact, Summary, Experience, Skills, Education
- **Validation Logic**: Field-level completeness checking per section requirements
- **AI Limitation**: 80% maximum score without AI usage, 100% with AI assistance
- **Live Updates**: Automatic recalculation on any CV data change via useEffect
- **Celebration System**: One-time 100% achievement celebration with Vietnamese messaging

State Management

- **CVData Interface**: Comprehensive TypeScript interface for type safety
- **Section Ordering**: Dynamic array-based section management with reordering support
- **Active Section Tracking**: UI state for highlighting currently editing section
- **Autosave Integration**: Prepared for autosave functionality with status indicators

Testing Architecture

- **Vitest Framework**: Modern testing with jsdom environment for React components
- **Score Testing**: 18 comprehensive tests covering all scoring scenarios
- **Type Safety**: Full TypeScript coverage with proper interface definitions
- **Mock Data**: Realistic Vietnamese CV data for development and testing

CV Guided Editing Export System

```

### Export Utilities (`src/utils/exportUtils.ts`)
**Purpose**: Comprehensive multi-format CV export system
**Dependencies**: html2canvas, jspdf, docx, file-saver
**Exports**: PDF (ATS-friendly), DOCX (MS Word), TXT (ATS-optimized)

**Key Functions**:
- `exportToPDF()` - High-quality PDF generation with html2canvas
- `exportToDOCX()` - Microsoft Word compatible document export
- `exportToTXT()` - ATS-optimized plain text export
- `generateFileName()` - Smart filename generation with timestamps
- `trackDownload()` - Export analytics tracking system
- `getExportAnalytics()` - Retrieve export analytics data

**Data Flow**:
...
CVDData → Export Functions → External Libraries → File Download
      ↓
      Analytics Tracking → localStorage
...

### Print Optimization (`src/styles/print.css`)
**Purpose**: Professional print styles for direct CV printing
**Features**: A4 page setup, typography optimization, element hiding
**Integration**: Imported globally in main.tsx

### PreviewPanel Enhancements
**New Features**:
- Multi-format export dropdown with descriptions
- Preview mode toggle for final review
- Direct print button with optimized styles
- Loading states and error handling UI
- Export analytics integration

**Component Dependencies**:
- exportUtils functions
- Lucide React icons (PrinterIcon, DownloadIcon, etc.)
- Enhanced state management for export operations

### Layout Architecture Improvements (June 2025)
- **Excel-style Freeze Panes**: Fixed header and preview panel implementation
  - **Fixed Header**: `position: fixed` with z-index hierarchy for always-visible navigation

```

```
- Fixed Preview Panel: Right-side panel frozen in place with responsive width distribution
- Single Scroller UX: Eliminated dual scrollers for Resume.io-like experience
- Main Page Scrolling: CV Editor content flows naturally with browser scroll
- Responsive Layout Distribution:
  - Large screens (1024px+): 50% editor / 50% preview
  - XL screens (1280px+): 55% editor / 45% preview
  - 2XL screens (1536px+): 60% editor / 40% preview
- Preview Scaling System: CSS zoom-based approach for complete CV display
  - Smart Scaling: Automatic zoom levels (0.65-0.75) to fit full CV in preview
  - WYSIWYG Experience: Consistent preview matching final PDF output
  - Container Preservation: Zoom affects content, not container dimensions

### UX Pattern Integration
- Job Description Analyzer: Moved from fixed header to CV Editor panel
  - Subtle Design: Gray background with minimal visual weight
  - Contextual Placement: Positioned after CV Score, before Contact Info
  - Collapsible Interface: Compact toggle with preview text
- Resume.io UX Standards: Professional layout patterns matching industry leader
  - Fixed Preview Panel: Never scrolls, always shows complete CV
  - Page Navigation: "1 / 1" indicator with navigation controls
  - Clean Visual Hierarchy: Minimal gray space, optimal content-to-chrome ratio

### Critical Issue Resolution System
- Import Failure Detection: Systematic debugging approach for blank screen issues
  - Binary Search Method: Component-by-component isolation testing
  - Dependency Chain Analysis: Testing imports at each level (Component → Hook → Utility → External)
  - Mock Implementation Strategy: Fallback systems for external dependencies
- Flexbox Scrolling Fixes: `min-h-0` solution for nested flex containers
  - Height Calculation Chain: Ensuring proper flexbox height propagation
  - Overflow Management: Strategic overflow properties for scrollable content
- Single Scroller Implementation: Eliminated confusing dual-scroll experience
  - Main Page Flow: Natural document scrolling with fixed preview panel
  - User Control: Single, intuitive scroll mechanism like Resume.io
```

```
### Performance Standards
```

```
Rendering Performance:
```

```
- **Initial Load:** < 2 seconds to interactive
- **Update Latency:** < 300ms for preview updates
- **Scroll Performance:** 60 FPS smooth scrolling
- **Memory Usage:** Efficient cleanup and management

**API Integration:**
- **Response Time:** < 1 second for AI suggestions
- **Error Handling:** Graceful degradation for API failures
- **Rate Limiting:** Smart throttling with user feedback
- **Caching:** Intelligent result caching for common requests

### Browser Compatibility

**Supported Browsers:**
- **Chrome:** Version 90+
- **Firefox:** Version 88+
- **Safari:** Version 14+
- **Edge:** Version 90+

**Feature Degradation:**
- **Drag & Drop:** Fallback to menu-based reordering
- **Real-time Preview:** Static preview with manual refresh
- **Advanced Typography:** Fallback fonts for unsupported browsers

### Accessibility Standards

**WCAG 2.1 AA Compliance:**
- **Keyboard Navigation:** Full functionality without mouse
- **Screen Reader Support:** Proper ARIA labels and structure
- **Color Contrast:** Minimum 4.5:1 ratio for all text
- **Focus Management:** Clear focus indicators and logical flow

**Assistive Technology:**
- **Voice Input:** Compatible with speech recognition
- **High Contrast:** Support for high contrast modes
- **Text Scaling:** Responsive to browser zoom settings
- **Motion Reduction:** Respect prefers-reduced-motion

---

## 🏗️ Quality Assurance & Testing
```

```

### Validation Testing

**Content Validation:**
- **Data Integrity:** Ensure no data loss during editing
- **Format Consistency:** Verify formatting across all sections
- **Export Accuracy:** Match preview output exactly
- **Character Encoding:** Handle special characters correctly

**User Experience Testing:**
- **Workflow Completion:** Test full CV creation process
- **Error Recovery:** Validate error handling and recovery
- **Performance Benchmarks:** Meet performance standards consistently
- **Cross-Device Testing:** Ensure consistent experience across devices

```

Requirements for a great CV for reference

Must-have sections

- Contact Information
 - Full Name (clearly visible, slightly larger font)
 - Professional Email Address (e.g., firstname.lastname@gmail.com)
 - Phone Number (formatted consistently, include country code if international)
 - Location (City, State, or Country clearly noted)
 - Professional LinkedIn URL (shortened and clickable)
 - GitHub or Portfolio URL (for software engineers, designers, etc.)
- Professional Summary or Objective
 - Concise statement (2–3 sentences)
 - Clearly communicates your expertise area
 - Tailored specifically to the targeted role/job description
 - Highlights key professional skills or achievements
 - Mentions quantifiable accomplishments where possible (metrics, numbers, percentages)
 - Reflects enthusiasm, confidence, and clarity of career direction
- Work Experience: Each role should clearly present:
 - Role/ Position title - Company Name
 - Accurate, industry-recognized titles
 - Clearly identifiable, avoid unnecessary abbreviations
 - If obscure, briefly describe sector (optional)
 - Employment Dates:
 - Start Date (Month Year) - End Date (Month Year) or Present (current role)
 - Reverse chronological order

- Key Achievements, not Responsibilities
 - 3–5 bullet points per role
 - Begin with powerful action verbs (e.g., Led, Managed, Achieved, Built, Developed, Optimized)
 - Clearly quantified impact and outcomes (%, \$, user growth, performance metrics)
 - Emphasize skills matching the job description (e.g., technology stack, teamwork, leadership)
 - Showcase progressive responsibility or leadership roles clearly
 - Highlight contributions to organizational goals or success stories
 - Mention cross-functional collaboration, mentorship, or leadership responsibilities
- Skills: Clearly organized into distinct categories:
 - **Technical Skills**
 - Programming languages clearly listed (Python, Java, JavaScript, etc.)
 - Frameworks/technologies explicitly named (React, Angular, Docker, AWS, Kubernetes). Preferably list down ones that listed in the job description/ requirements
 - Databases explicitly noted (SQL, PostgreSQL, MongoDB, etc.)
 - **Tools & Platforms**
 - Git, Jira, Jenkins, CI/CD pipelines clearly specified
 - Cloud platforms like AWS, Azure, Google Cloud clearly indicated
 - **Soft Skills**
 - Teamwork, communication, leadership, problem-solving, etc.
 - **Certifications & Additional Competencies**
 - Professional certifications (AWS Certified Solutions Architect, PMP, etc.)
 - Date of certification or expected expiration (if applicable)
 - Credential ID or verification link (if applicable)
- Education
 - Degree type (e.g., Bachelor of Science, Master of Science)
 - Specific field of study or major
 - Institution name (university or college clearly specified)
 - Graduation year
 - Academic distinctions (Honors, Summa Cum Laude, etc.) (if applicable)
 - Relevant coursework (if applicable)
 - Projects or Thesis title, if relevant to your targeted role (especially for technical roles or recent grads) (if applicable)
- Volunteer Experience: a must for fresh-graduate CVs
 - Clearly named organization or cause
 - Specific dates involved
 - Key achievements, outcomes or activities (highlight soft skills like leadership, teamwork, organizational skills)

Others

- Must be 1 page
- Must NOT have an avatar

With this comprehensive specification, the product/design/engineering team can proceed to implement the Guided Editing interface in Figma and code (e.g., using Cursor or similar IDEs) with a clear understanding of the desired functionality and user experience. All components, from draggable section cards to suggestion modals, should be designed to work together seamlessly, delivering a resume-building experience that is intuitive, powerful, and truly AI-first. The result will be a resume editor that not only matches but greatly improves upon Resume.io's model, setting a new standard for what users can expect when crafting their CVs.

Out of scope

Tone customization

Tone Adjustment: We have pre-defined style prompts for each tone. E.g., formal: no contractions, more authoritative wording; friendly: a bit more warm language, maybe a first-person if allowed, etc. We apply that to the text.

- **Rephrasing and Tone:** We use AI models to rephrase text. Likely the same underlying model but given a smaller snippet and instruction ("rephrase professionally"). The logic ensures not to change factual content (or if it might, we verify with user).

AI strength control

If the user's bullet says "Improved X", an AI might ask "By how much?" – this interactive query is an advanced feature that really engages the user to provide their data. We might implement it as part of "Improve bullet": if AI thinks a metric is needed, it could pop a mini modal or form for the user to input a number or result, then incorporate it. This way, it remains factual.